



INSTITUTO POLITÉCNICO INDUSTRIAL DE LUANDA
ÁREA DE FORMAÇÃO DE INFORMÁTICA
CURSO TÉCNICO DE INFORMÁTICA

JOÃO STÉLVIO RIBEIRO CARIQUI

PROJECTO TECNOLÓGICO

**SISTEMA DE GESTÃO DE PRONTUÁRIOS ELECTRÔNICOS
PARA O HOSPITAL MILITAR CENTRAL**



INSTITUTO POLITÉCNICO INDUSTRIAL DE LUANDA
ÁREA DE FORMAÇÃO DE INFORMÁTICA
CURSO TÉCNICO DE INFORMÁTICA

JOÃO STÉLVIO RIBEIRO CARIQUI

**SISTEMA DE GESTÃO DE PRONTUÁRIOS ELECTRÔNICOS
PARA O HOSPITAL MILITAR CENTRAL**

Trabalho de conclusão de curso apresentado ao Instituto Politécnico Industrial de Luanda – Área de Informática, como requisito parcial para obtenção do título de Técnico Médio, Curso de Técnico de Informática sob as orientações do Orientador: Prof. Eng.º Lucas Pazito Miguel Abel.

Prof. Eng.º Lucas Pazito Miguel Abel
(Prof. Orientador)

LUANDA, 2025

EPÍGRAFE

"Todo mundo nesse país deveria aprender a programar... Porque isso ensina você a pensar." - (Steve Job)

"Nenhum saber é saber completo!" - (Desconhecido)

"O segredo da mudança é concentrar toda a sua energia não em lutar contra o velho, mas em construir o novo." - Sócrates

"A informação é o óleo do século 21, e a análise é o motor de combustão." - Peter Sondergaard

DEDICATÓRIA

Dedico este projeto à minha família, cujo amor e apoio incondicional foram essenciais durante toda a nossa jornada acadêmica. Agradeço especialmente aos meus pais, que sempre acreditaram em mim e incentivaram-me a alcançar os meus sonhos.

Aos meus amigos e colegas, que compartilharam comigo momentos de desafios e conquistas, proporcionando um ambiente de colaboração e incentivo mútuo.

À equipe administrativa e aos professores do Instituto Politécnico Industrial de Luanda, que inspiraram este projeto com seu compromisso constante com a excelência educacional e o bem-estar dos alunos.

AGRADECIMENTO

Gostaria de expressar a minha sincera gratidão a todas as pessoas que contribuíram para a realização deste projeto. Em primeiro lugar, gostaria de agradecer à equipe administrativa e aos funcionários da secretaria do Instituto Politécnico Industrial de Luanda (IPIL) por sua colaboração e apoio ao longo de todo o processo de desenvolvimento deste sistema.

Também quero estender os meus agradecimentos aos nossos orientadores e professores, cuja orientação e expertise foram fundamentais para o sucesso deste projeto. Seus insights valiosos e feedbacks construtivos foram essenciais para orientar nosso trabalho e alcançar resultados significativos.

Além disso, gostaria de agradecer aos meus colegas e amigos que forneceram apoio moral e encorajamento durante os momentos desafiadores deste projeto. Suas palavras de incentivo foram uma fonte de inspiração e motivação ao longo do caminho.

Por fim, não posso deixar de expressar a minha profunda gratidão aos alunos e demais partes interessadas que participaram dos testes e forneceram feedback valioso sobre o sistema desenvolvido. Suas contribuições foram inestimáveis para aprimorar a usabilidade e funcionalidade do sistema.

Agradeço a todos os envolvidos por seu comprometimento e dedicação, que foram fundamentais para o sucesso deste projeto. Seu apoio foi verdadeiramente inestimável e sou imensamente grato por fazer parte deste trabalho conjunto.

O meu muito obrigado!

RESUMO

O presente estudo é desenvolvido com o objetivo de implementar uma tecnologia para gestão de prontuários eletrônicos. Esta tecnologia consiste em criar uma relação entre os pacientes e os seus prontuários digitais, permitindo o registro, armazenamento e consulta de informações clínicas de forma eficiente e segura. A cada paciente será atribuído um prontuário eletrônico específico e único, que irá facilitar os profissionais de saúde no acompanhamento e histórico médico de cada indivíduo.

Esta tecnologia de gestão irá permitir:

- O registro eletrônico dos pacientes e atribuição de um prontuário digital a cada um deles;
- Facilitar o acesso e atualização das informações médicas;
- Melhorar a segurança e confidencialidade dos dados de saúde;
- Permitir a consulta rápida e integrada das informações clínicas.

Palavras-chave: Implementar, Gestão, prontuário eletrônico, pacientes, tecnologia, segurança, saúde.

ABSTRACT

The present study is developed with the aim of implementing a technology for electronic medical record management. This technology consists of creating a connection between patients and their digital records, allowing for the efficient and secure registration, storage, and consultation of clinical information. Each patient will be assigned a specific and unique electronic medical record, facilitating healthcare professionals in monitoring and maintaining the medical history of each individual.

This management technology will enable:

- The electronic registration of patients and the assignment of a digital medical record to each of them;
- Facilitating access to and updating of medical information;
- Improving the security and confidentiality of health data;
- Allowing quick and integrated consultation of clinical information.

Keywords: Implement, Management, Electronic Medical Record, Patients, Technology, Security, Health.

LISTA DE FIGURAS

Figura 1 - Logo do HTML.....	28
Figura 2 – Logo do CSS	29
Figura 3 - Exemplo de código em CSS	29
Figura 4 - Exemplo de código em JavaScript.....	33
Figura 5 – Logo do TypeScript.....	33
Figura 6 - Exemplo de Herança em Typescript	34
Figura 7 - Exemplo de Encapsulamento em Typescript	35
Figura 8 - Exemplo de abstração em Typescript	36
Figura 9 - Exemplo de Polimorfismo em Typescript	38
Figura 10 – Logo do React	41
Figura 11 – Logo do TailwindCSS.....	45
Figura 12 – Processo de requisições de um servidor NodeJS	49
Figura 13 - Imagem do Prontuário Electrónico.....	60
Figura 14 - Princípio fundamental da DSDM.....	65
Figura 15 - Ciclo de Vida de um Projeto em DSDM	66
Figura 16 – Tela de visualização do prontuário do paciente	70
Figura 17 – Diagrama de caso de uso do Sistema	74
Figura 18 – Diagrama da base de dados	75
Figura 19 – Protótipo do sistema.....	83
Figura 20 - Logo do SQL	85
Figura 21 - Logo do JavaScript	87
Figura 22 - Logo React Native	87
Figura 23 - Logo do Node Js	88
Figura 24 - Logo do PHP	88
Figura 25 - Logo do MySql	89
Figura 26 - Logo do Node Js	89
Figura 27 - Logo do JSX	90
Figura 28 - Logo do WAMP.....	90
Figura 29 – Página inicial para medicos	93
Figura 30 – Página inicial para pacietes	93
Figura 31 – Tela de Cadastro.....	100
Figura 32 – Tela de Login	100
Figura 33 – Tela de Agendamento de consulta.....	101
Figura 34 – Tela de Perfil.....	101

LISTA DE TABELAS

Tabela 1 - Requisitos funcionais do aplicativo.....	73
Tabela 2 - Requisitos não funcionais identificados	74
Tabela 3 - Tabela do cronograma de atividades	98
Tabela 4 – Orçamento do projecto.....	99

LISTA DE ABREVIATURAS, SIGLAS E ACRÓNIMOS

DML	Data Manipulation Language
DLL	Dynamic Link library
MAC	Media Access Control
APP	Mobile Application
RUP	Rational Unified Process
UML	Unified Modeling Language
XML	Extensible Markup Language
HTML	HyperText Markup Language
SQL	Structured Query Language
API	Application Programming Interface
DER	Diagrama Entidade Relacional
UI	User Interface
MVC	Model View Controller MVC
MVVM	Model View View Model
WSDL	Web Services Description Language
REST	Representational transfer protocol
HTTP	Hypertext Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
SGBD	Sistema Gerenciador de Banco de Dados
SGA	Sistema de Gestão Acadêmica
CSS	Cascading Style Sheets

ÍNDICE

INTRODUÇÃO.....	1
PROBLEMÁTICA.....	2
JUSTIFICATIVA.....	3
OBJECTIVO GERAL.....	4
OBJETIVOS ESPECÍFICOS.....	4
ESTRUTURA DO TRABALHO	5
1. FUNDAMENTAÇÃO TEÓRICA.....	7
1.1. GESTÃO DA INFORMAÇÃO MEDICA	7
1.2. TECNOLOGIAS DE INFORMAÇÃO E COMUNICAÇÃO (TICS) NA MEDICINA.....	8
1.3. SISTEMAS DE INFORMAÇÃO DE INFORMAÇÃO PARA HOSPITAIS	9
1.4. SEGURANÇA DE INFORMAÇÃO NA MEDICINA	11
1.5. AUTOMAÇÃO DE PROCESSOS ADMINISTRATIVOS	12
1.6. VANTAGENS EM AUTOMATIZAR PROCESSOS ADMINISTRATIVOS.....	13
1.7. AUTOMATIZAÇÃO DE PROCESSOS ADMINISTRATIVOS	14
1.8. ETAPAS PARA IMPLEMENTAÇÃO E AUTOMATIZAÇÃO DE PROCESSOS	15
1.9. LEGISLAÇÃO E NORMAS OBRIGATÓRIAS	15
2. PROGRAMAÇÃO WEB.....	19
2.1. HISTÓRIA DA INTERNET.....	19
2.2. INTRODUÇÃO À WEB	20
2.3.1. COMO FUNCIONA UMA APLICAÇÃO WEB	21
2.3.2. DIMENSÕES DE UM PROJECTO WEB.....	22
2.3. APLICAÇÃO WEB VS WEBSITE	22
2.3.1. VANTAGENS DE UMA APLICAÇÃO WEB	24
2.3.1. ETAPAS DO DESENVOLVIMENTO DE UMA APLICAÇÃO WEB	25
2.4. TECNOLOGIAS E LINGUAGENS DE DESENVOLVIMENTO WEB.....	26
2.4.1. PRINCIPAIS TECNOLOGIAS USADAS NO DESENVOLVIMENTO WEB	27
2.4.2. TECNOLOGIAS DO FRONT-END.....	40
2.4.4. TECNOLOGIAS DO BACK-END.....	46
3. BANCO DE DADOS	53
3.1. TIPOS DE BANCO DE DADOS	53

3.1.1. BANCOS DE DADOS RELACIONAIS (RDBMS):	53
3.1.2. BANCOS DE DADOS NÃO-RELACIONAIS (NoSQL):	53
3.1.3. BANCOS DE DADOS DE GRAFOS	54
3.1.4. BANCOS DE DADOS EM MEMÓRIA	54
3.2. SGBD.....	54
3.3. VANTAGENS DE USAR BANCO DE DADOS	55
3.4. PHP	56
3.4.1. CARACTERÍSTICAS.....	56
3.5. ORMs.....	56
3.5.1. Eloquent ORM	56
3.6. Doctrine ORM.....	57
4. METODOLOGIAS DE DESENVOLVIMENTO.....	60
4.1. PRONTUÁRIO ELÊTRONICO	60
4.1.1. USO E TIPOS DE PRONTUÁRIOS.....	60
4.2. UML.....	61
4.3 METODOLOGIAS ÁGEIS	61
4.3.1. O DSDM.....	62
5 – ANÁLISE E DESENHO DO SISTEMA E TECNOLOGIAS USADAS.....	70
5.1. VISUALIZAÇÃO DO PRONTRUÁRIO DO PACIENTE	70
5.1.1. FUNCIONALIDADES PRINCIPAIS	70
5.2. PROCESSO DE IMPLEMENTAÇÃO	71
5.2.1. ANÁLISE DE REQUISITOS	71
5.2.2. DESIGN DA APLICAÇÃO.....	71
5.2.3. DESENVOLVIMENTO E TESTES.....	71
5.2.4. SEGURANÇA DA INFORMAÇÃO.....	72
5.2.5. IMPLANTAÇÃO.....	72
5.3. BENEFÍCIOS DA APLICAÇÃO.....	72
5.4. ANÁLISE E DESENHO DO SISTEMA	72
5.5. ANÁLISE DE REQUISITOS.....	72
5.5.1. REQUISITOS FUNCIONAIS	73
5.5.2. REQUISITOS NÃO FUNCIONAIS.....	73
5.6. DIAGRAMA DE CASOS DE USO	74
5.7. BASE DE DADOS	75

5.7.1. DESCRIÇÕES PRINCIPAIS.....	75
5.8. NORMALIZAÇÃO	83
5.9. PROTÓTIPO DO SISTEMA.....	84
5.10. TECNOLOGIAS USADAS.....	85
5.10.1. SQL	85
5.10.2. JavaScript	87
5.10.3. React Native	87
5.10.4. CSS (Cascading Style Sheets).....	88
5.10.7. Node Js	88
5.10.8. Express.js.....	88
5.10.9. MySQL.....	89
5.10.10. Outras tecnologias	89
5.11. PADRÕES DE ARQUITETURAS DE SOFTWARES	91
5.11.1. MVC.....	91
6. RESULTADOS E RECOMENDAÇÕES	93
6.1 RESULTADOS.....	93
6.2. RECOMENDAÇÕES	94
APÊNDICE A – TABELA DO CRONOGRAMA DAS ACTIVIDADES	98
APÊNDICE B – ORÇAMENTO.....	99
APÊNDICE C – FORMULÁRIOS	100
REFERÊNCIAS BIBLIOGRÁFICAS.....	102

INTRODUÇÃO

A implementação de uma tecnologia para gestão de prontuários eletrônicos, consiste em um sistema de digitalização e centralização das informações medicas dos pacientes. Este sistema tem o objectivo primordial de diminuir o trabalho dos médicos no controle das informações medicas dos pacientes.

Este sistema ou tecnologia é desenvolvida com base alguns conceitos de programação. A presente tecnologia é desenvolvida em linguagens web e de programação como HTML, CSS, Javascript.

Prontuário Electrónico: é um sistema digital utilizado para registrar, armazenar e gerenciar informações médicas dos pacientes de forma segura e eficiente. Ele permite o acesso rápido ao histórico clínico, exames, diagnósticos e tratamentos, facilitando a comunicação entre os profissionais de saúde e melhorando a qualidade do atendimento. O prontuário eletrônico substitui os registros em papel, garantindo maior organização, integridade e confidencialidade dos dados.

PROBLEMÁTICA

Os hospitais enfrentam dificuldades na gestão das informações médicas dos pacientes devido a atual forma de conservação das informações médicas dos pacientes, que é ineficiente, desorganizada e não oferece segurança total. Elas precisam de um sistema de gestão rápido, confiável e seguro para garantir que as informações médicas dos pacientes sejam conservadas corretamente e que os as informações médicas permaneçam protegidas.

JUSTIFICATIVA

Com a “implementação de uma tecnologia para gestão de prontuários eletrônicos”, os hospitais poderão melhorar sua eficiência na gestão de informações, garantir maior controle sobre o movimento dos documentos médicos dos pacientes no ambiente hospitalar e reduzir custos de registro manual.

Implementar uma tecnologia para gestão de prontuários eletrônicos, irá diminuir e facilitar o trabalho dos médicos no processo de comunicação entre os médicos e controle das alterações feitas nos documentos médicos de cada paciente.

OBJECTIVO GERAL

- Criar um sistema de gestão de prontuários mais seguro e eficiente para médicos e pacientes que seja capaz de mantê-los informados de forma rápida e confiável;
- Criar um sistema que limite e facilite o trabalho dos médicos no controle das informações médicas dos pacientes.

OBJETIVOS ESPECÍFICOS

- Desenvolver um sistema de gestão de prontuários eletrônicos para conservar e centralizar as informações médicas dos pacientes com maior precisão, confiabilidade e rapidez;
- Criar um banco de dados que permita o armazenamento de informações médicas dos pacientes em uma única fonte e que seja seguro e fácil de acessar.
- Implementar funcionalidades de controle de acesso e permissões por perfil de usuário (como médicos, recepcionistas e administradores) para garantir a confidencialidade dos dados.
- Integrar funcionalidades de busca e filtragem de prontuários por nome, data, especialidade ou outros critérios relevantes, facilitando o acesso rápido às informações clínicas.

ESTRUTURA DO TRABALHO

O corrente trabalho está constituído por 5 capítulos com os conteúdos que se seguem:

Capítulo I – Este capítulo apresenta informações sobre gestão de informações medicas, tecnologias de informação, segurança e automação dos dados entre outros.

Capítulo II – Este capítulo são demonstrados e mencionados conteúdos relacionados ao desenvolvimento web.

Capítulo III – Este capítulo aborda sobre conceitos relacionados a banco de dados.

Capítulo IV – Este capítulo agrega assuntos relacionados a metodologias de desenvolvimento.

Capítulo V – Este capítulo é o principal, onde abordado sobre a análise e desenho do sistema implementado.

Capítulo VI – Este capítulo apresenta resultados da implementação e recomendações consideráveis.



CAPITULO I

FUNDAMENTAÇÃO TEÓRICA

1. FUNDAMENTAÇÃO TEÓRICA

1.1. GESTÃO DA INFORMAÇÃO MÉDICA

Uma administração eficaz da **informação médica** é fundamental para o bom funcionamento de hospitais e clínicas, pois possibilita o registro, organização e análise de dados referentes aos pacientes, atendimentos, profissionais da saúde e demais atividades assistenciais e administrativas. Além de simplificar as tarefas burocráticas, essa gestão bem-feita também desempenha um papel crucial na melhoria da experiência do paciente e na elevação da qualidade do atendimento médico. Abaixo estão alguns pontos essenciais sobre a relevância da administração da informação médica e como sistemas integrados podem otimizar esses procedimentos:

- ❖ **Tomada de Decisão com Base em Dados:** Uma boa gestão da informação médica proporciona aos gestores e profissionais da saúde acesso a dados precisos e atualizados sobre histórico de atendimentos, diagnósticos, exames, tratamentos e indicadores clínicos. Com essas informações, é possível tomar decisões mais embasadas e estratégicas, melhorar os protocolos de atendimento, distribuir recursos com mais eficiência e identificar áreas críticas que necessitam de melhorias.
- ❖ **Aumento da Eficiência Operacional:** Sistemas integrados de gestão médica automatizam diversas tarefas administrativas e clínicas, como agendamento de consultas, cadastro de pacientes, emissão de receitas, solicitação de exames e controle de estoque de medicamentos. Isso reduz a carga de trabalho da equipe, minimiza erros e retrabalho e permite que os profissionais se concentrem em atividades mais estratégicas e voltadas ao cuidado do paciente.
- ❖ **Aprimoramento da Experiência do Paciente:** Um sistema de gestão médica eficiente oferece ao paciente acesso facilitado a informações importantes, como horários de consultas, resultados de exames, receitas e histórico clínico. Além disso, permite que ele realize agendamentos online, receba notificações e acompanhe o progresso do seu tratamento. Essa acessibilidade e transparência contribuem para uma experiência mais acolhedora, segura e satisfatória.

- ❖ **Fomento à Colaboração e Comunicação:** Sistemas integrados de informação médica facilitam a comunicação entre médicos, enfermeiros, técnicos, recepcionistas e outros profissionais da saúde. Por meio de recursos como prontuários eletrônicos, mensagens internas e compartilhamento de documentos clínicos, esses sistemas promovem a troca de informações em tempo real, fortalecendo a colaboração interdisciplinar e melhorando a qualidade do atendimento.
- ❖ **Segurança e Integridade dos Dados:** Um sistema integrado centraliza e protege os dados médicos dos pacientes, garantindo sua segurança e integridade. Recursos como controle de acesso, criptografia, autenticação de usuários e backups regulares ajudam a prevenir vazamentos e acessos indevidos, assegurando que as informações sensíveis dos pacientes sejam tratadas de forma ética e conforme as legislações de privacidade, como a Lei Geral de Proteção de Dados (LGPD).

Portanto, a implementação de **sistemas integrados de gestão da informação médica** não apenas otimiza os processos administrativos e clínicos, mas também promove uma experiência mais eficiente, transparente e segura para todos os envolvidos na área da saúde. Esses sistemas são indispensáveis para acompanhar as demandas crescentes da medicina moderna e atender às expectativas dos pacientes quanto à qualidade, agilidade e humanização dos serviços de saúde.

1.2. TECNOLOGIAS DE INFORMAÇÃO E COMUNICAÇÃO (TICS) NA MEDICINA

As Tecnologias de Informação e Comunicação (TICs) desempenham um papel fundamental na modernização da gestão em saúde, permitindo que hospitais, clínicas e demais instituições médicas otimizem seus processos administrativos e assistenciais, além de melhorar a eficiência operacional e a qualidade do atendimento. O uso de sistemas de informação em saúde é uma das formas mais eficazes de integrar as TICs na gestão médica, automatizando tarefas rotineiras e simplificando operações clínicas complexas. Abaixo, são apresentados alguns pontos-chave sobre o papel das TICs na modernização da gestão médica:

1. Automação de Processos Administrativos e Clínicos: Sistemas de informação em saúde permitem a automação de uma ampla gama de processos, como agendamento de consultas, prontuário eletrônico, controle de estoque, emissão de receitas, faturamento e controle de exames laboratoriais. Essa automação reduz significativamente o tempo e o esforço dedicados a tarefas repetitivas, liberando os profissionais da saúde para atividades de maior complexidade e impacto clínico.

2. Centralização e Organização de Dados Clínicos: As TICs possibilitam a centralização e organização de informações médicas dos pacientes, como histórico clínico, resultados de exames, medicações prescritas e procedimentos realizados. Essa centralização facilita o acesso rápido e seguro a dados relevantes, promovendo uma tomada de decisão mais ágil, informada e baseada em evidências por parte de médicos e gestores.

3. Melhoria da Comunicação e Colaboração Interdisciplinar: As TICs facilitam a comunicação entre médicos, enfermeiros, técnicos, recepcionistas e demais profissionais da saúde, por meio de plataformas online, sistemas internos de mensagens, prontuários compartilhados e alertas automatizados. Isso promove o trabalho em equipe, reduz erros de comunicação e melhora a coordenação do cuidado, beneficiando diretamente o paciente.

4. Acesso Remoto e Telemedicina: Com o avanço das TICs, tornou-se possível realizar atendimentos médicos a distância por meio da telemedicina e de plataformas de monitoramento remoto. Isso amplia o acesso aos serviços de saúde, especialmente em regiões remotas ou com carência de especialistas, além de permitir o acompanhamento contínuo de pacientes crônicos no conforto de seus lares.

5. Personalização do Atendimento e Medicina Preditiva: Sistemas baseados em TICs podem ser usados para monitorar sinais vitais, analisar dados clínicos e gerar relatórios personalizados sobre o estado de saúde de cada paciente. Com base nesses dados, é possível desenvolver planos de tratamento individualizados e até prever riscos futuros, como crises ou complicações, possibilitando uma medicina mais preventiva, personalizada e eficaz.

As TICs têm se tornado essenciais na transformação digital da área da saúde, proporcionando mais eficiência, agilidade, precisão e segurança aos processos clínicos e administrativos. O uso inteligente dessas tecnologias contribui significativamente para a melhoria dos serviços prestados, promovendo um sistema de saúde mais moderno, integrado e centrado no paciente.

1.3. SISTEMAS DE INFORMAÇÃO PARA HOSPITAIS E CLÍNICAS

Existem diversos **sistemas de informação voltados para hospitais, clínicas e unidades de saúde**, cada um com funcionalidades específicas que contribuem para a eficiência administrativa, segurança do paciente e qualidade do atendimento. Abaixo estão alguns dos sistemas mais comuns utilizados nessas instituições:

1. Sistema de Gestão Hospitalar (SGH ou HIS - Hospital Information System):

- **Funcionalidades:** Gerenciamento de prontuários eletrônicos, agendamento de consultas e exames, cadastro de pacientes, controle de internações e altas, emissão de receitas e atestados, gestão de leitos e procedimentos.

- **Benefícios:** Centralização das informações clínicas, agilidade nos processos assistenciais, redução de erros médicos, melhora na tomada de decisões clínicas e administrativas.

2. Sistema de Prontuário Eletrônico do Paciente (PEP ou EMR - Electronic Medical Record):

- **Funcionalidades:** Registro detalhado do histórico médico do paciente, resultados de exames, evoluções médicas, prescrições, anotações de enfermagem e controle de alergias e medicações.

- **Benefícios:** Acesso rápido e seguro às informações clínicas, continuidade do cuidado, redução do uso de papel, aumento da segurança do paciente e apoio à medicina baseada em evidências.

3. Sistema de Gestão de Laboratório (LIS - Laboratory Information System):

- **Funcionalidades:** Solicitação e rastreamento de exames, integração com equipamentos laboratoriais, emissão de laudos, controle de qualidade de amostras e organização dos fluxos laboratoriais.

- **Benefícios:** Agilidade na entrega de resultados, rastreabilidade das análises, redução de erros e retrabalhos, melhor organização do laboratório.

4. Sistema de Gestão Financeira Hospitalar :

- **Funcionalidades:** Controle de faturamento hospitalar (convênios e particular), emissão de boletos, controle de contas a pagar e receber, integração com o setor de compras, geração de relatórios financeiros e contábeis.

- **Benefícios:** Automatização dos processos financeiros, controle preciso das receitas e despesas, suporte à auditoria e conformidade fiscal, redução de inadimplência.

5. Sistema de Gestão de Recursos Humanos (RH Médico-Hospitalar):

- **Funcionalidades** Registro de dados de profissionais da saúde e administrativos, controle de escalas e plantões, folha de pagamento, gestão de benefícios, controle de ponto e avaliações de desempenho.

- **Benefícios:** Otimização da gestão de equipes, conformidade com legislações trabalhistas e normas da saúde, melhor alocação de profissionais e valorização do capital humano.

6. Sistema de Gestão de Farmácia Hospitalar:

- **Funcionalidades:** Controle de estoque de medicamentos e materiais, dispensação por paciente, rastreabilidade de lotes, controle de validade, integração com prescrições eletrônicas.

- **Benefícios** Redução de perdas e desperdícios, garantia de segurança na administração de medicamentos, controle rigoroso do uso e melhor gestão de recursos farmacêuticos.

7. Sistema de Business Intelligence (BI) em Saúde:

- **Funcionalidades:** Análise de indicadores clínicos, operacionais e financeiros, criação de dashboards gerenciais, identificação de gargalos e oportunidades de melhoria.

- **Benefícios** Tomada de decisão baseada em dados, melhoria na performance institucional, apoio à gestão estratégica e à acreditação hospitalar.

Esses sistemas são fundamentais para a eficiência operacional, segurança dos pacientes e qualidade no atendimento médico. Sua integração e uso inteligente contribuem diretamente para a construção de um sistema de saúde mais moderno, seguro e centrado no paciente.

1.4. SEGURANÇA DE INFORMAÇÃO NA MEDICINA

A segurança da informação na área da saúde é uma prioridade crítica, considerando a sensibilidade dos dados médicos armazenados em hospitais, clínicas e demais instituições. Informações de pacientes, como diagnósticos, exames, históricos clínicos e prescrições, exigem proteção rigorosa contra acessos não autorizados, vazamentos e ataques cibernéticos. Abaixo são apresentados os principais desafios e as boas práticas para garantir a segurança da informação na medicina:

Desafios:

1. Crescente Volume de Dados Sensíveis: Instituições de saúde armazenam uma grande quantidade de **dados clínicos e pessoais** dos pacientes, incluindo informações sobre doenças, tratamentos, alergias, histórico familiar, dados financeiros e convênios. Esse volume crescente aumenta a complexidade e o risco de vazamentos ou acessos indevidos.

2. Variedade de Ameaças Cibernéticas: O setor da saúde está entre os mais visados por cibercriminosos. As instituições enfrentam ameaças como **ransomware, phishing, malware, acesso não autorizado e roubo de identidade médica**. Um ataque bem-sucedido pode comprometer todo o sistema, prejudicar atendimentos e gerar danos éticos, legais e financeiros.

3. Integração e Interoperabilidade de Sistemas: A necessidade de integrar diferentes sistemas (laboratórios, farmácias, clínicas externas) pode aumentar os pontos de vulnerabilidade. A interoperabilidade, embora benéfica para o fluxo de informações, também exige mecanismos de segurança mais avançados.

4. Baixa Conscientização dos Profissionais de Saúde: Médicos, enfermeiros e demais profissionais muitas vezes não recebem treinamento suficiente em **boas práticas de segurança digital**, o que pode levar a erros como senhas fracas, compartilhamento indevido de dados ou abertura de links maliciosos.

Melhores Práticas:

1. Políticas de Segurança da Informação: Desenvolver e implementar políticas de segurança da informação claras e abrangentes que definam as responsabilidades dos usuários, procedimentos de segurança, requisitos de senha, entre outros aspectos.

2. Treinamento em Conscientização em Segurança: Realizar programas regulares de treinamento em conscientização em segurança para todos os membros da comunidade medicinal, destacando os riscos de segurança cibernética e as melhores práticas para proteger os dados.

3. Implementação de Medidas Técnicas de Segurança: Adotar medidas técnicas de segurança, como firewalls, antivírus, criptografia de dados, autenticação de dois fatores e monitoramento de rede, para proteger os sistemas e dados contra ameaças cibernéticas.

4. Atualizações e Patches de Segurança: Manter todos os sistemas, aplicativos e dispositivos atualizados com as últimas correções de segurança e patches de software para mitigar vulnerabilidades conhecidas.

5. Controle de Acesso e Privacidade de Dados: Implementar controles de acesso rigorosos para proteger os dados dos alunos contra acessos não autorizados e garantir conformidade com regulamentações de privacidade, como o GDPR (Regulamento Geral de Proteção de Dados) na União Europeia.

6. Plano de Resposta a Incidentes: Desenvolver e testar regularmente um plano de resposta a incidentes para lidar eficazmente com violações de segurança cibernética, incluindo procedimentos de notificação de violação e recuperação de dados.

Ao adotar essas melhores práticas e enfrentar os desafios relacionados à segurança da informação na medicina, os hospitais podem proteger efetivamente os dados dos pacientes e médicos, garantir a integridade dos sistemas e cumprir as regulamentações de privacidade em constante evolução.

1.5. AUTOMAÇÃO DE PROCESSOS ADMINISTRATIVOS

A automatização de processos administrativos refere-se a utilização de tecnologias para realizar tarefas que eram feitas manualmente. Porém, para dar sequência a explicação, nós precisamos que o termo processos administrativos esteja bem estabelecido.

Basicamente, ele pode ser definido como sequências de tarefas realizadas com o objetivo de atender às necessidades dos clientes de um negócio.

Podemos dividir os processos administrativos em três tipos principais:

- primários (focados em entregar valor ao cliente);

- de suporte (auxiliam nas variadas demandas de trabalho);
- de gerenciamento (servem para acompanhar, medir e controlar os processos com um ponto de vista).

A automatização dos processos administrativos é feita por softwares de gerenciamento de tarefas, sistemas de informação e ferramentas de análise. Como objetivos principais podemos citar a redução do tempo gasto na execução dos processos administrativos e a melhora da organização.

Dessa forma, por meio da tecnologia, é possível aperfeiçoar a integração entre sistemas e setores, promovendo diversas vantagens para a gestão e consequentemente aos clientes.

1.6. VANTAGENS EM AUTOMATIZAR PROCESSOS ADMINISTRATIVOS

Ao atribuir um trabalho manual a uma ferramenta digital, teremos dois benefícios principais. Em primeiro lugar, agilidade nos processos e, em segundo, uma equipe com maior flexibilidade de tempo para lidar com outras demandas.

Mas, não é só isso. Há diversas outras vantagens que valem a pena ser citadas. Elas incluem:

1. Redução de erros: ao automatizar tarefas manuais, a possibilidade de erros humanos é reduzida significativamente, o que aumenta a precisão e a confiabilidade dos processos administrativos;
2. Eficiência operacional: a automatização de processos permite que as tarefas sejam executadas de forma mais rápida e eficiente, diminuindo assim o tempo necessário para concluir as atividades;
3. Redução de custos: a eliminação de tarefas que não agregam valor pode ajudar a reduzir os custos operacionais, como os gastos relacionados com as impressões e a necessidade de materiais de escritório, por exemplo;
4. Maior segurança dos dados: as soluções de automatização conseguem controlar e auditar diversas atividades, ajudando a garantir a segurança dos dados e a manter a conformidade com regulamentações e normas;

5. Melhor colaboração: a automatização de processos permite uma melhor colaboração entre as equipes, pois as informações são compartilhadas de forma mais rápida e eficiente;
6. Melhor experiência do cliente: ao ter mais eficiência e controle nos processos administrativos, as empresas podem oferecer aos seus clientes um serviço mais rápido e de maior qualidade.
7. Melhor armazenamento: este item, está ligado à questão de maior segurança de dados, ao evitar a perda e o extravio de documentos. Um bom armazenamento também está relacionado com uma melhor organização de todas as informações.

1.7. AUTOMATIZAÇÃO DE PROCESSOS ADMINISTRATIVOS

Quando o assunto é gestão de dados, subentende-se que há um grande volume de informações a serem processadas. Nesse sentido, a falta de uma boa ferramenta tecnológica para administração pode impactar negativamente na performance do negócio.

Por esta razão, os serviços de automatização e informação se tornaram tão necessários nesses últimos anos. Eles representam uma ferramenta de suporte para gerenciar e dominar o excesso de dados direcionados aos sistemas digitais.

Imagine uma empresa onde várias pessoas são responsáveis pela gestão das notas fiscais. As chances dela enfrentar dificuldades para localizar os documentos, e até perdê-los, são muito altas, pois os arquivos não estão organizados e armazenados no mesmo lugar.

Utilizar ferramentas de automatização também é benéfico nos seguintes aspectos:

- integração de informações;
- fácil localização de documentos;
- ampla visualização de dados;
- diminuição de processos burocráticos como demora na validação de compras, comprovação de documentos, por exemplo.

Além do gerenciamento de notas fiscais, a automatização de processos administrativos pode entregar excelentes resultados quando empregada na gestão de vendas, atendimento ao cliente, comércio eletrônico, entre outros.

1.8. ETAPAS PARA IMPLEMENTAÇÃO E AUTOMATIZAÇÃO DE PROCESSOS

Depois de tudo isso, podemos estar nos perguntando por onde começar, não é? Pensando nisso, nós enumeramos algumas para poder ter noção, finalmente e começar organizar:

1. Entenda quais setores da sua empresa (instituição) necessitam da automatização;
2. Faça um mapeamento minucioso de todas as etapas do processo a ser automatizado;
3. Escolha o software ideal para suas necessidades;
4. Identifique pontos de melhoria na execução desse processo;
5. Coloque a satisfação do aluno como foco da automação de processos;
6. Crie um cronograma contendo todas as etapas e prazos necessários para que a implementação seja concluída;
7. Programe treinamentos para que a equipe se familiarize com o sistema e as mudanças;
8. Acompanhe os resultados obtidos com a automatização do processo;
9. Faça ajustes sempre que detectar algum problema ou necessidade de reparação.

1.9. LEGISLAÇÃO E NORMAS OBRIGATÓRIAS

Analisar as leis e regulamentações relevantes para a gestão de dados acadêmicos é fundamental para garantir a conformidade legal e proteger a privacidade dos alunos. Abaixo estão algumas das principais leis e regulamentações que podem impactar a gestão de dados acadêmicos:

1. Lei Geral de Proteção de Dados (LGPD) - Brasil:

- A LGPD, inspirada no Regulamento Geral de Proteção de Dados (GDPR) da União Europeia, estabelece regras sobre coleta, armazenamento, tratamento e compartilhamento de dados pessoais. Ela se aplica a todas as organizações que lidam com dados pessoais de residentes

brasileiros, incluindo instituições de ensino. A LGPD define direitos dos titulares dos dados, como o direito de acesso, correção e exclusão de dados, e exige o consentimento explícito para o tratamento de dados pessoais.

2. Family Educational Rights and Privacy Act (FERPA) - Estados Unidos:

- A FERPA é uma lei federal que protege a privacidade dos registros educacionais dos alunos. Ela estabelece direitos dos pais e alunos maiores de 18 anos sobre o acesso e a divulgação de registros educacionais, garantindo a confidencialidade dessas informações. As instituições de ensino devem obter o consentimento dos alunos ou pais antes de divulgar informações educacionais e implementar medidas de segurança para proteger os registros educacionais.

3. General Data Protection Regulation (GDPR) - União Europeia:

- O GDPR é uma regulamentação da União Europeia que estabelece regras sobre proteção de dados e privacidade para indivíduos na UE. Ele se aplica a todas as organizações que processam dados pessoais de residentes da UE, incluindo instituições de ensino. O GDPR define direitos dos titulares dos dados, como o direito de acesso, retificação, exclusão e portabilidade dos dados, e requer o consentimento explícito para o processamento de dados pessoais.

4. Leis Nacionais de Proteção de Dados Pessoais em outros países:

- Muitos países têm suas próprias leis nacionais de proteção de dados pessoais, que estabelecem requisitos específicos para o tratamento de dados pessoais. Por exemplo, no Canadá, a Lei de Privacidade das Informações Pessoais e Eletrônicas (PIPEDA) estabelece regras sobre a coleta, uso e divulgação de informações pessoais por organizações no setor privado.

Além dessas leis específicas de proteção de dados, as instituições de ensino também podem ser afetadas por outras regulamentações relacionadas à gestão de dados acadêmicos, como leis de acesso à informação, legislação trabalhista e regulamentações educacionais específicas do país.

Portanto, é essencial que as instituições de ensino estejam cientes das leis e regulamentações aplicáveis à gestão de dados acadêmicos em sua jurisdição e implementem medidas adequadas de segurança e conformidade para proteger a privacidade dos alunos e cumprir suas obrigações legais. Isso pode incluir a designação de um encarregado de proteção de dados, a implementação de políticas de privacidade e segurança da informação, a realização de treinamento

em conscientização de privacidade e a adoção de medidas técnicas para proteger os dados pessoais dos alunos.



CAPITULO II

PROGRAMAÇÃO WEB

2. PROGRAMAÇÃO WEB

2.1. HISTÓRIA DA INTERNET

A internet surgiu a partir de pesquisas militares no auge da Guerra Fria. Neste contexto, em que os dois blocos ideológicos e politicamente antagônicos exerciam enorme controle e influência no mundo, qualquer mecanismo, qualquer inovação, qualquer ferramenta nova poderia contribuir nessa disputa liderada pela União Soviética e pelos Estados Unidos: as duas superpotências compreendiam a eficácia e a necessidade absoluta dos meios de comunicação.

Nessa perspectiva, o governo dos Estados Unidos temia um ataque russo às bases militares. Um ataque poderia trazer ao público informações sigilosas, tornando os EUA vulneráveis. Então foi idealizado um modelo de troca e compartilhamento de informações que permitisse a descentralização das mesmas. Assim, se o Pentágono fosse atingido, as informações armazenadas ali não estariam perdidas.

Era preciso, portanto, criar uma rede, a ARPANET, criada pela ARPA, sigla para Advanced Research Projects Agency. Em 1962, J. C. R. Licklider, do Instituto Tecnológico de Massachusetts (MIT), já falava em termos da criação de uma Rede Intergaláctica de Computadores (Intergalactic Computer Network, em inglês).[carece de fontes] A Internet também teve outros importantes atores que influenciaram o seu surgimento, dentre eles: professores universitários (como Ken King), estudantes (como Vint Cerf), empresas de tecnologia (como a IBM) e alguns políticos norte-americanos (como Al Gore); caindo-se, portanto, a tese que vigorava anteriormente que enfatizava somente a vertente militar da sua criação.

A ARPANET funcionava através de um sistema conhecido como chaveamento de pacotes, que é um sistema de transmissão de dados em rede de computadores no qual as informações são divididas em pequenos pacotes, que por sua vez contém trecho dos dados, o endereço do destinatário e informações que permitiam a remontagem da mensagem original.

O ataque inimigo nunca aconteceu, mas o que o Departamento de Defesa dos Estados Unidos não sabia era que dava início ao maior fenômeno midiático do século XX', único meio de comunicação que em apenas 4 anos conseguiria atingir cerca de 50 milhões de pessoas.

Em 29 de Outubro de 1969 ocorreu a transmissão do que pode ser considerado o primeiro E-mail da história. O texto desse primeiro e-mail seria "LOGIN", conforme desejava o Professor Leonard Kleinrock da Universidade da Califórnia em Los Angeles (UCLA), mas o computador no Stanford Research Institute, que recebia a mensagem, parou de funcionar após receber a letra "O".

A partir de 1982, o uso da ARPANET tornou-se maior no âmbito acadêmico. Inicialmente o uso era restrito nos EUA, mas se expandiu para outros países, como Holanda, Dinamarca e Suécia. Desde então, começou a ser utilizado o nome internet.

Em 1992 começaram a surgir diversas empresas provedoras de acesso à internet naquele país. No mesmo ano, o laboratório Europeu de Física de Partículas (CERN), inventou a World Wide Web (www), que começou a ser utilizada para colocar informações ao alcance de qualquer usuário da internet.

Desde então, a difusão da rede foi enorme. Hoje, a internet tem mais de 250 milhões de usuários em todo o mundo. Até o final de 2024, o tráfego mundial de e-mails deverá estar em torno de 35 bilhões de mensagens diárias.

2.2. INTRODUÇÃO À WEB

A palavra web pertence ao vocabulário da língua inglesa e significa “Rede”. Mesmo pertencendo ao léxico inglês o termo é amplamente utilizado em português como muitos outros com a mesma origem. Este conceito é utilizado dentro do espaço de tecnologia para referir-se a um sistema de interligação de documentos e recursos de forma geral, através da internet. O Termo tanto pode se referir a uma página de internet, a um lugar virtual (website) ou a um servidor. Este documento eletrônico contém informações adaptadas ao espaço em que se encontram de forma que possam ser acessadas por usuários do mundo todo que utilizam um determinado navegador (browser).

2.3. APLICAÇÃO WEB

Uma aplicação web. Trata-se de uma aplicação de software que roda na internet, em vez de funcionar com base em sistemas operacionais.

Assim, é um sistema com funcionalidades completas, que foi programado a partir de requisitos e dos princípios da engenharia de software. Contudo, seu grande diferencial é que ele é

feito para funcionar na internet. O que permitiu o surgimento de plataformas cada vez mais robustas na web foi, justamente, a computação em nuvem. Ela mudou a forma de pensar aplicações, de um paradigma de produtos para um novo, focado em serviços.

Nesse caso, as aplicações funcionam como um serviço, oferecendo alguma função importante para os usuários. Como exemplo, podemos mencionar o e-mail, os sites de redes sociais e os sites de editores de texto. Alguns até oferecem um pagamento por demanda, de acordo com o uso, e se encaixam no paradigma SaaS (software como um serviço). Nesse caso, é preciso ter uma assinatura com uma conta para visualizar as informações e executar as funções necessárias.

O sucesso de uma empresa passa pelos sistemas que ela utiliza no seu dia-a-dia para realizar o seu trabalho da melhor maneira possível, então pensando nisso, entender o conceito de uma aplicação web é fundamental.

2.3.1. COMO FUNCIONA UMA APLICAÇÃO WEB

Uma aplicação web funciona com base na infraestrutura da internet. Por isso, é preciso entender, primeiro, como funciona a web.

O site fica armazenado em um servidor. Quando o usuário solicita acesso por meio de um endereço URL, ele automaticamente consegue conexão com o servidor DNS (servidor com uma lista de domínios) que, por sua vez, acessa o IP que referencia aquele site.

Então, o site envia informações de download para o usuário. Por fim, o usuário pode interagir com a plataforma enviando informações, alterando e salvando novos dados etc. Tomemos como exemplo quando acessamos o site de uma rede social. Inicialmente, enviamos uma requisição para abrir a página. O servidor encontra o endereço e responde com as informações para download.

Em seguida, nos deparamos com um formulário de login. Ao preencher as informações e enviar, o sistema vai checar o banco de dados em que estão as informações de todos os usuários cadastrados. Uma vez chegado, o sistema indica se houve um match com o que já existe ou não.

Caso haja um match, a aplicação permite o acesso e solicita ao servidor as informações da página principal com os dados específicos daquela conta. Logo, este pacote é enviado para o download do usuário. Caso não haja um match, o sistema volta na mesma página do formulário de login e indica um erro. Uma vez que já está logado, o usuário consegue realizar ações, como:

publicar uma imagem, interagir em posts dos amigos, adicionar novos amigos, etc. Tudo isso representa interações que podem ser traduzidas em envio de requisições cliente-servidor.

2.3.2. DIMENSÕES DE UM PROJECTO WEB

Sobre as dimensões de um projecto web (aplicação web), nós podemos dividir em 3 dimensões:

- Estrutural: Fala da disposição de informações principais gerenciadas pela aplicação;
- Navegacional: Cuida de aspectos das experiências e de como as informações serão acessadas;
- De representação: Lida com a forma de como as informações serão mostradas e como o acesso será definido.

É preciso pensar em cada um desses fatores no projecto de uma aplicação web.

2.3. APLICAÇÃO WEB VS WEBSITE

No ambiente web é muito comum que alguns termos geram muitas dúvidas, mesmo entre aqueles que convivem diariamente com esses ambientes, seja no trabalho ou nos estudos. Algumas dessas dúvidas são a diferença entre um web site e uma aplicação web, até mesmo programadores costumam interpretar erroneamente esses conceitos.

Embora os termos "aplicação web" e "website" sejam frequentemente usados de forma intercambiável, há distinções importantes entre os dois conceitos. Aqui está uma comparação entre aplicação web e website:

Eis as diferenças entre eles:

→ Website

Estrutura Estática: Um website tradicional é geralmente composto por páginas estáticas, onde o conteúdo é fixo e não muda com frequência. Os websites são frequentemente usados para fornecer informações estáticas sobre uma empresa, organização, produto ou serviço.

Navegação Simples: A navegação em um website geralmente segue uma estrutura simples de links, permitindo aos usuários acessar diferentes páginas de conteúdo. Os websites podem incluir menus de navegação, barras laterais e rodapés para facilitar a exploração do conteúdo.

Objetivo Informativo ou Promocional: O principal objetivo de um website é fornecer informações sobre uma empresa, produto ou serviço, ou promover uma marca ou iniciativa. Os websites são frequentemente utilizados como uma ferramenta de marketing para atrair e informar clientes potenciais.

Interatividade Limitada: Embora alguns websites possam incluir elementos interativos, como formulários de contato ou caixas de pesquisa, a interatividade é geralmente limitada em comparação com as aplicações web.

→ **Aplicação Web:**

Funcionalidade Dinâmica: Uma aplicação web é uma plataforma interativa que oferece funcionalidades dinâmicas e interativas aos usuários. Em vez de simplesmente apresentar informações, uma aplicação web permite aos usuários realizar ações específicas, como enviar formulários, fazer compras online ou colaborar em documentos compartilhados.

Interação em Tempo Real: As aplicações web podem fornecer interações em tempo real com os usuários, como atualizações automáticas de conteúdo, notificações instantâneas e bate-papo ao vivo. Isso cria uma experiência mais dinâmica e envolvente para os usuários.

Personalização e Conta do Usuário: Muitas aplicações web oferecem recursos de personalização e contas de usuário, permitindo aos usuários criar perfis, armazenar preferências e acessar conteúdo personalizado com base em suas necessidades e interesses individuais.

Processamento de Dados e Lógica de Negócios: As aplicações web podem realizar processamento de dados complexo e executar lógica de negócios do lado do servidor, permitindo funcionalidades avançadas, como cálculos complexos, análise de dados em tempo real e integração com sistemas de backend.

2.3.1. VANTAGENS DE UMA APLICAÇÃO WEB

Algumas das vantagens de uma aplicação web:

- **Acessibilidade Global:** Uma aplicação web pode ser acessada de qualquer lugar com conexão à internet, permitindo que os usuários acessem o sistema a partir de uma variedade de dispositivos, como computadores, tablets e smartphones.
- **Não Requer Instalação:** Como uma aplicação web é acessada através de um navegador da web, não é necessário instalar software adicional nos dispositivos dos usuários. Isso simplifica o processo de implantação e atualização, pois todas as mudanças são implementadas no servidor central.
- **Compatibilidade Multiplataforma:** As aplicações web são desenvolvidas utilizando tecnologias padrão da web, como HTML, CSS e JavaScript, garantindo compatibilidade com diferentes sistemas operacionais e navegadores. Isso proporciona uma experiência consistente para os usuários, independentemente do dispositivo ou sistema que estão utilizando.
- **Facilidade de Atualização:** As atualizações em uma aplicação web são implementadas centralmente no servidor, garantindo que todos os usuários tenham acesso imediato às últimas versões do software. Isso elimina a necessidade de atualizações individuais em cada dispositivo e garante que todos os usuários estejam utilizando a versão mais recente da aplicação.
- **Integração com Serviços Web:** As aplicações web podem facilmente integrar-se com serviços web e APIs, permitindo a troca de dados e a interoperabilidade com outros sistemas e plataformas online. Isso facilita a integração com serviços de terceiros, como redes sociais, sistemas de pagamento e serviços de armazenamento na nuvem.
- **Escalabilidade:** Uma aplicação web pode ser dimensionada facilmente para lidar com um grande número de usuários e volume de dados, simplesmente aumentando os recursos do servidor. Isso proporciona uma maior flexibilidade para as empresas, que podem expandir sua capacidade de forma rápida e eficiente à medida que crescem.

- **Custo Reduzido:** As aplicações web muitas vezes apresentam custos iniciais mais baixos em comparação com o desenvolvimento de aplicativos nativos para diferentes plataformas (iOS, Android, etc.). Isso ocorre porque uma única aplicação web pode ser acessada em várias plataformas sem a necessidade de desenvolvimento separado para cada uma delas.

Além disso, os custos de manutenção de uma aplicação web também podem ser mais baixos, já que as atualizações são feitas centralmente no servidor e não exigem o envio de atualizações para cada dispositivo individualmente.

2.3.1. ETAPAS DO DESENVOLVIMENTO DE UMA APLICAÇÃO WEB

O desenvolvimento de uma aplicação web geralmente segue várias etapas, desde a concepção inicial até o lançamento e manutenção contínua. Aqui estão as etapas comuns do desenvolvimento de uma aplicação web:

2.3.1.1. DEFINIÇÃO DE OBJETIVOS E REQUISITOS

A primeira etapa do desenvolvimento de uma aplicação web é definir os objetivos do projeto e os requisitos funcionais e não funcionais. Isso envolve identificar o propósito da aplicação, o público-alvo, os recursos necessários e as restrições do projeto.

2.3.1.2. PLANEJAMENTO E ANÁLISE

Nesta etapa, a equipe de desenvolvimento realiza uma análise detalhada dos requisitos do projeto e elabora um plano de desenvolvimento. Isso pode incluir a criação de documentos de especificação, cronogramas de trabalho, alocação de recursos e definição de tecnologias a serem utilizadas.

2.3.1.3. DESIGN DE INTERFACE DE USUÁRIO (UI/UX)

O design da interface de usuário (UI) e da experiência do usuário (UX) é uma parte crucial do desenvolvimento de uma aplicação web. Isso envolve a criação de wireframes, protótipos e designs visuais que são intuitivos, atraentes e eficientes em termos de usabilidade.

2.3.1.4. DESENVOLVIMENTO FRONT-END

A etapa de desenvolvimento front-end envolve a criação da interface de usuário da aplicação web usando tecnologias como HTML, CSS e JavaScript, ReactJs, VueJs, Angular, Svelte, Astro, Remix, NextJs etc. Isso inclui a implementação de layouts, estilos, interações e funcionalidades visuais que os usuários vêem e interagem diretamente.

2.3.1.5. DESENVOLVIMENTO BACK-END

O desenvolvimento back-end é responsável pela implementação da lógica de negócios, gerenciamento de dados e integração com sistemas externos. Isso geralmente envolve o uso de linguagens de programação como Python, Java, PHP ou Node.js, e frameworks como Django, Spring, Laravel, NestJs, AdonisJs ou Express.js.

2.3.1.6. TESTES

Os testes são uma parte essencial do desenvolvimento de uma aplicação web para garantir que ela funcione conforme o esperado e seja livre de erros e falhas. Isso inclui testes de unidade, testes de integração, testes de sistema e testes de aceitação do usuário.

2.3.1.7. IMPLANTAÇÃO

Uma vez que a aplicação web tenha sido desenvolvida e testada com sucesso, ela está pronta para ser implantada em um ambiente de produção como Vercel, Netlify, Heroku etc. Isso geralmente envolve configurar servidores, bancos de dados e outros recursos necessários para hospedar e executar a aplicação.

2.4. TECNOLOGIAS E LINGUAGENS DE DESENVOLVIMENTO WEB

O desenvolvimento web é o processo de criação e manutenção de aplicações e páginas web acessíveis através da internet ou de uma intranet. Ele engloba uma variedade de tarefas e disciplinas, desde a criação do conteúdo visual até o desenvolvimento da lógica de negócios por trás de uma aplicação web. O objetivo principal do desenvolvimento web é fornecer uma experiência de usuário agradável e funcional, independentemente do dispositivo ou navegador utilizado.

Uma das características mais importantes do desenvolvimento web é a capacidade de criar páginas interativas e dinâmicas, onde os usuários podem interagir com o conteúdo e realizar ações, como preencher formulários, fazer compras online, enviar mensagens, entre outras atividades. Isso é possível graças ao uso de linguagens de marcação, estilização e programação que trabalham em conjunto para criar uma experiência coesa e envolvente para o usuário final.

Além da criação de interfaces visuais, o desenvolvimento web também envolve a implementação de lógica de negócios no lado do servidor, lidando com operações como autenticação de usuários, processamento de formulários, acesso a bancos de dados e muito mais. Isso é geralmente feito utilizando-se linguagens de programação específicas e frameworks que facilitam o desenvolvimento de aplicações web robustas e escaláveis.

Outro aspecto importante do desenvolvimento web é a garantia de que as aplicações sejam acessíveis a uma ampla variedade de dispositivos e navegadores. Isso é alcançado através da criação de layouts responsivos e do uso de práticas de desenvolvimento que garantam uma boa experiência do usuário, independentemente do tamanho da tela ou das capacidades do dispositivo utilizado para acessar a aplicação.

Isso tudo só é possível graças às tecnologias que são usadas no desenvolvimento Web. Abaixo vamos detalhar cada uma delas, pelo menos as principais.

2.4.1. PRINCIPAIS TECNOLOGIAS USADAS NO DESENVOLVIMENTO WEB

2.4.1.1. HTML

HTML, ou Hypertext Markup Language, é a linguagem de marcação padrão para criação e estruturação de páginas web. É uma linguagem de marcação que utiliza tags para definir a estrutura e o conteúdo de uma página web. As tags são elementos que envolvem e descrevem diferentes partes do conteúdo, como texto, imagens, links e elementos multimídia. HTML fornece a base para a construção de páginas web e é essencial para qualquer desenvolvedor web.

Até ao momento o HTML ainda não é considerado uma linguagem de programação oficialmente, porque ainda não consegue funcionalidades dinâmicas, apenas estáticas como: parágrafos, seções, botões etc.



Figura 1 - Logo do HTML

2.4.1.1.1. FUNCIONAMENTO

Marcação Estrutural: HTML é usado para estruturar o conteúdo de uma página web. Ele define os elementos básicos da página, como cabeçalhos, parágrafos, listas, links e imagens, utilizando tags específicas. Por exemplo, `<h1>` é usado para definir um cabeçalho de primeiro nível (heading 1), `<p>` é usado para definir um parágrafo e `<img>` é usado para inserir uma imagem.

As três principais partes de um elementos HTML são:

Tag de abertura – Usada para dizer onde começa um elemento. A tag é cercada por sinais de menor e maior (`<tag>`).

Tag de fechamento – Usada para dizer onde termina um elemento. A tag é cercada por sinais de menor, barra e maior (`</tag>`).

Exemplo final: `<h1>Hello World</h1>`

2.4.1.2. CSS

Cascading Style Sheets (abreviado CSS) é um mecanismo para adicionar estilos (cores, fontes, espaçamento, etc.) a uma página web, aplicado diretamente nas tags HTML ou ficar contido dentro das tags `<style>`. Também é possível adicionar estilos adicionando um link para um arquivo CSS que contém os estilos. Assim, quando se quiser alterar a aparência dos documentos vinculados a este arquivo CSS, basta modificá-lo.

Com a variação de atualizações dos navegadores, o suporte ao CSS pode variar. A interpretação dos navegadores pode ser avaliada com o teste Acid2, que se tornou uma forma base

de revelar quão eficiente é o suporte de CSS, fazendo com que a nova versão em desenvolvimento do Firefox seja totalmente compatível a ele, assim como o Opera já é. O Doctype informado, ou a ausência dele, determina o quirks mode ou o strict mode, modificando o modo como o CSS é interpretado e a página desenhada.



Figura 2 – Logo do CSS

2.4.1.2.1. RESPONSABILIDADE

O CSS pode ser usado para criar layouts responsivos que se adaptam a diferentes tamanhos de tela e dispositivos. Isso é alcançado através de técnicas como media queries, que permitem aplicar estilos com base nas características do dispositivo, como largura da tela, orientação e densidade de pixels.

```
<!-- Inline CSS Styles -->
<h1 style="font-weight: bold; font-size: 42px;">
  I will be bold and 42 pixels!
</h1>
<p style="color: blue;">I will be blue!</p>
```

Figura 3 - Exemplo de código em CSS

2.4.1.2.2. VANTAGENS DO CSS

CSS, ou Cascading Style Sheets, oferece uma série de vantagens significativas no desenvolvimento web. Aqui estão algumas das principais vantagens do CSS:

Separação de Conteúdo e Estilo: Uma das vantagens mais importantes do CSS é a capacidade de separar o conteúdo da apresentação. Isso significa que você pode criar estilos em

um arquivo CSS separado e aplicá-los a várias páginas HTML. Essa separação facilita a manutenção do código, permite a reutilização de estilos e torna o código mais limpo e organizado.

Flexibilidade e Controle de Estilo: CSS oferece uma ampla gama de opções de estilo, permitindo controlar praticamente todos os aspectos da aparência visual de uma página web. Você pode definir cores, fontes, tamanhos, espaçamento, layouts, efeitos visuais e muito mais, proporcionando flexibilidade total no design da interface do usuário.

Eficiência no Desenvolvimento: Usar CSS pode tornar o processo de desenvolvimento web mais eficiente e produtivo. Uma vez que você definiu seus estilos em um arquivo CSS separado, pode aplicá-los a várias páginas HTML sem a necessidade de repetir o código de estilo em cada página. Isso economiza tempo e esforço, especialmente em projetos grandes e complexos.

Facilidade de Manutenção: Com a separação de conteúdo e estilo, a manutenção do código torna-se muito mais fácil. Se você precisar fazer alterações nos estilos de um site, pode simplesmente editar o arquivo CSS correspondente, em vez de ter que modificar cada página HTML individualmente. Isso ajuda a garantir consistência e coesão em todo o site.

Compatibilidade entre Navegadores: CSS é amplamente suportado por todos os navegadores modernos, o que significa que os estilos que você define em CSS serão exibidos de forma consistente em diferentes navegadores e plataformas. Isso reduz a necessidade de hacks e soluções alternativas para lidar com inconsistências de renderização entre navegadores.

Responsividade e Adaptação a Dispositivos: CSS permite criar layouts responsivos que se adaptam a diferentes tamanhos de tela e dispositivos. Com o uso de técnicas como media queries, você pode criar estilos específicos para dispositivos móveis, tablets e desktops, proporcionando uma experiência de usuário consistente em todos os dispositivos.

2.4.1.3. JAVASCRIPT

JavaScript (frequentemente abreviado como JS) é uma linguagem de programação interpretada estruturada, de script em alto nível com tipagem dinâmica fraca e multiparadigma (protótipos, orientado a objeto, imperativo e funcional). Juntamente com HTML e CSS, o JavaScript é uma das três principais tecnologias da World Wide Web. JavaScript permite páginas da Web interativas e, portanto, é uma parte essencial dos aplicativos da web. A

grande maioria dos sites usa, e todos os principais navegadores têm um mecanismo JavaScript dedicado para executá-lo. É atualmente a principal linguagem para programação client-side em navegadores web. É também bastante utilizada do lado do servidor através de ambientes como o node.js.

Como uma linguagem multiparadigma, o JavaScript suporta estilos de programação orientados a eventos, funcionais e imperativos (incluindo orientado a objetos e prototype-based), apresentando recursos como fechamentos (closures) e funções de alta ordem comumente indisponíveis em linguagens populares como Java e C++. Possui APIs para trabalhar com texto, matrizes, datas, expressões regulares e o DOM, mas a linguagem em si não inclui nenhuma E/S, como instalações de rede, armazenamento ou gráficos, contando com isso no ambiente host em que está embutido.

Foi originalmente implementada como parte dos navegadores web para que scripts pudessem ser executados do lado do cliente e interagem com o usuário sem a necessidade deste script passar pelo servidor, controlando o navegador, realizando comunicação assíncrona e alterando o conteúdo do documento exibido, porém os mecanismos JavaScript agora estão incorporados em muitos outros tipos de software host, incluindo em servidores e bancos de dados da Web e em programas que não são da Web, como processadores de texto e PDF, e em tempo de execução ambientes que disponibilizam JavaScript para escrever aplicativos móveis e de desktop, incluindo widgets de área de trabalho.

Os termos Vanilla JavaScript e Vanilla JS se referem ao JavaScript não estendido por qualquer estrutura ou biblioteca adicional. Scripts escritos em Vanilla JS são códigos JavaScript simples.

Embora existam semelhanças entre JavaScript e Java, incluindo o nome da linguagem, a sintaxe e as respectivas bibliotecas padrão, as duas linguagens são distintas e diferem muito no design; JavaScript foi influenciado por linguagens de programação como Self e Scheme.

É baseada em ECMAScript, padronizada pela Ecma International nas especificações ECMA-262 e ISO/IEC 16262.

2.4.1.3.1. FUNCIONAMENTO

JavaScript é uma linguagem de programação de alto nível, interpretada e orientada a objetos, utilizada principalmente no desenvolvimento web para adicionar interatividade e dinamismo às páginas. Aqui está um resumo do funcionamento do JavaScript:

Integração com HTML e CSS: JavaScript é frequentemente usado em conjunto com HTML e CSS para criar páginas web dinâmicas e interativas. Ele pode ser incluído em documentos HTML usando a tag `<script>` ou pode ser importado de arquivos externos.

Execução no Navegador: JavaScript é executado no navegador do cliente (ou seja, no computador ou dispositivo do usuário). Quando uma página web contendo código JavaScript é carregada, o navegador interpreta e executa o código JavaScript para manipular o conteúdo, a estrutura e o estilo da página.

Manipulação do DOM: Uma das principais funções do JavaScript é manipular o DOM (Document Object Model), que representa a estrutura hierárquica de elementos HTML em uma página. JavaScript pode ser usado para acessar, modificar, adicionar e remover elementos do DOM, permitindo a criação de páginas web dinâmicas e responsivas.

Eventos: JavaScript permite responder a eventos do usuário, como cliques do mouse, pressionamentos de tecla, submissões de formulários e movimentos do mouse. Você pode definir manipuladores de eventos para responder a esses eventos e executar ações específicas quando eles ocorrem.

Programação Assíncrona: JavaScript suporta programação assíncrona, o que significa que você pode realizar operações que levam tempo (como fazer requisições HTTP para o servidor) sem bloquear a execução do código. Isso é feito usando callbacks, Promises ou `async/await` para lidar com operações assíncronas de forma eficiente.

APIs do Navegador: JavaScript fornece acesso a uma variedade de APIs do navegador, que permitem interagir com recursos do navegador e do sistema operacional. Isso inclui APIs para manipular elementos de mídia (como áudio e vídeo), acessar a geolocalização do usuário, armazenar dados localmente no navegador (usando Web Storage ou IndexedDB), e muito mais.

Frameworks e Bibliotecas: Existem muitos frameworks e bibliotecas JavaScript disponíveis que simplificam o desenvolvimento de aplicativos web complexos. Alguns exemplos populares incluem React.js, AngularJS, Vue.js e jQuery etc.

2.5.1.3.2. Exemplo de código

```
// Numero aleatorio de 0 a 9
const number = Math.floor(Math.random() * 10)

//If e else tradicional
if (number % 2 === 0) {
  console.log('Par')
} else {
  console.log('Ímpar')
}

//If ternário
number % 2 === 0 ? console.log('Par') : console.log('Ímpar')
```

Figura 4 - Exemplo de código em JavaScript

2.4.1.3.3. TYPESCRIPT

O TypeScript é uma linguagem de programação desenvolvida pela Microsoft que se baseia no JavaScript adicionando recursos de tipagem estática opcional, interfaces, enums e outros recursos de linguagem que facilitam o desenvolvimento de aplicativos em larga escala.



Figura 5 – Logo do TypeScript

2.4.1.3.3.1 PRINCIPAIS CONCEITOS DO TYPESCRIPT

HERANÇA

Em programação orientada a objetos (POO), a herança é um conceito fundamental que permite que uma classe (conhecida como classe filha ou subclasse) herda atributos e métodos de outra classe (conhecida como classe pai ou superclasse). Isso significa que a classe filha pode reutilizar e estender o comportamento da classe pai, evitando a redundância de código e promovendo a reutilização.

A classe filha herda todos os membros (atributos e métodos) da classe pai e pode adicionar novos membros ou modificar o comportamento dos membros existentes. A herança permite criar hierarquias de classes, onde classes mais especializadas podem herdar características de classes mais gerais.

```
class Animal {
  nome: string;

  move(distancia: number = 0) {
    console.log(`${this.nome} moveu ${distancia}m.`);
  }
}

class Dog extends Animal {
  bark() {
    console.log(`${this.nome} latiu.`);
  }
}
```

Figura 6 - Exemplo de Herança em Typescript

ENCAPSULAMENTO

Encapsulamento é um dos princípios fundamentais da programação orientada a objetos (POO) e refere-se à prática de agrupar os dados (atributos) e os métodos que operam nesses dados em uma única unidade chamada de classe. Essa unidade é responsável por controlar o acesso aos dados, determinando quem pode ler e modificar os valores dos atributos.

Em POO, os atributos de uma classe são frequentemente declarados como privados, o que significa que eles só podem ser acessados e modificados pelos próprios métodos da classe. Isso impede o acesso direto aos dados por partes do programa que não fazem parte da classe, fornecendo assim um nível de segurança e controle sobre o estado interno do objeto.

```
class Carro {  
    private velocidade: number;  
  
    constructor(velocidade: number) {  
        this.velocidade = velocidade;  
    }  
}
```

Figura 7 - Exemplo de Encapsulamento em Typescript

ABSTRAÇÃO

A abstração em programação orientada a objetos (POO) é o processo de identificar as características essenciais de um objeto do mundo real e representá-las em um modelo de software de forma simplificada, focando apenas nos detalhes relevantes para o problema em questão e omitindo os detalhes irrelevantes. Isso permite que os desenvolvedores criem modelos de software que se assemelham aos objetos e conceitos do mundo real, facilitando a compreensão e a manipulação do sistema.

A abstração é alcançada através do uso de classes e objetos. Uma classe é uma abstração de um conjunto de objetos que compartilham características comuns. Essas características são representadas pelos atributos da classe. Além disso, uma classe pode definir métodos que descrevem o comportamento dos objetos dessa classe.

```

abstract class Shape {
    abstract calculateArea(): number;
}

class Circle extends Shape { radius:
    number;

    constructor(radius: number) {
        super();
        this.radius = radius;
    }

    calculateArea(): number {
        return Math.PI * this.radius ** 2;
    }
}

class Rectangle extends Shape { width:
    number;
    height: number;

    constructor(width: number, height: number) {
        super();
        this.width = width;
        this.height = height;
    }
}

```

Figura 8 - Exemplo de abstração em Typescript

POLIMORFISMO

Polimorfismo é outro conceito fundamental da programação orientada a objetos (POO). Refere-se à capacidade de um objeto de uma classe se comportar de maneira diferente dependendo do contexto em que é utilizado. Em outras palavras, o polimorfismo permite que objetos de

diferentes classes sejam tratados de maneira uniforme, desde que compartilhem uma interface comum.

Existem duas formas principais de polimorfismo:

Polimorfismo de sobrescrita (Override): Nesse caso, uma subclasse redefine (sobrescrever) um método da classe pai, fornecendo uma implementação específica para a subclasse. Isso permite que um método com o mesmo nome seja chamado em objetos diferentes e, dependendo do tipo do objeto em tempo de execução, a versão correta do método é executada.

Polimorfismo de sobrecarga (Overload): Apesar de não ser tão comum em linguagens orientadas a objetos como Python, o polimorfismo de sobrecarga refere-se à capacidade de uma classe ter múltiplos métodos com o mesmo nome, mas com diferentes parâmetros. A decisão sobre qual método chamar é tomada em tempo de compilação, com base na quantidade e nos tipos dos argumentos passados para o método.

```

class Animal { name: string;

    constructor(name: string) {
        this.name = name;
    }

    makeSound(): void {
        console.log("Some generic sound");
    }
}

class Dog extends Animal {
    makeSound(): void {
        console.log("Woof! Woof!");
    }
}

class Cat extends Animal {
    makeSound(): void {
        console.log("Meow! Meow!");
    }
}

function makeAnimalSound(animal: Animal): void {
    animal.makeSound();
}

```

Figura 9 - Exemplo de Polimorfismo em Typescript

2.4.1.3.3.2. VANTAGENS DO TYPESCRIPT:

Tipagem Estática Opcional: TypeScript permite a definição de tipos de dados para variáveis, parâmetros de função, retornos de função e outros elementos do código. Isso proporciona maior segurança e confiabilidade durante o desenvolvimento, pois ajuda

a detectar erros de tipo em tempo de compilação, reduzindo bugs e melhorando a qualidade do código.

Orientação a Objetos: TypeScript suporta programação orientada a objetos com classes, interfaces, herança, polimorfismo, entre outros conceitos. Isso facilita a criação de código mais organizado, reutilizável e fácil de manter, especialmente em projetos grandes e complexos.

Fácil Integração com JavaScript: Como o TypeScript é uma superset do JavaScript, todo código JavaScript existente é válido em um arquivo TypeScript.

Isso significa que você pode começar a usar TypeScript gradualmente em projetos existentes, adicionando tipagem estática e outros recursos à medida que necessário, sem a necessidade de reescrever todo o código.

Ferramentas de Desenvolvimento Avançadas: TypeScript é frequentemente usado em conjunto com ferramentas de desenvolvimento poderosas, como o Visual Studio Code da Microsoft, que oferece suporte integrado para TypeScript, incluindo realce de sintaxe, verificação de tipos em tempo real, refatoração de código e muito mais. Isso melhora significativamente a produtividade do desenvolvedor.

Ecossistema Forte: TypeScript tem uma comunidade de desenvolvedores ativa e um ecossistema de ferramentas e bibliotecas em constante crescimento. Muitas bibliotecas populares do JavaScript, como React, Angular e Vue, têm suporte oficial ou não oficial para TypeScript, permitindo que você aproveite os benefícios da tipagem estática em seus projetos.

Manutenção do Código Simplificada: Com tipagem estática e outras características de linguagem, TypeScript torna a manutenção do código mais fácil e menos propensa a erros. A documentação gerada automaticamente e a capacidade de navegar rapidamente pelo código facilitam a compreensão e a colaboração em projetos de equipe.

Suporte ao Next.js: Next.js é um framework de React para construção de aplicativos web e páginas estáticas ou dinâmicas. Ele é frequentemente usado em projetos que exigem renderização do lado do servidor (SSR) ou renderização estática (SSG).

TypeScript é uma escolha popular para desenvolvedores que utilizam Next.js devido à sua forte tipagem estática, que ajuda a evitar erros de código e melhora a qualidade do aplicativo.

Next.js tem suporte nativo para TypeScript, o que significa que você pode começar a usar TypeScript em um projeto Next.js sem configuração adicional. O Next.js fornece tipos de dados para suas APIs e componentes, facilitando o desenvolvimento de aplicativos com TypeScript.

Além disso, o Next.js oferece integração perfeita com ferramentas de desenvolvimento como o Visual Studio Code, que fornece recursos avançados de edição, depuração e verificação de tipos para projetos Next.js com TypeScript.

2.4.2. TECNOLOGIAS DO FRONT-END

Para que uma página web ou website funcione é necessário o processamento de dois lados: lado do cliente (navegador) e o lado do servidor, que hospeda o site. O primeiro lado trata naturalmente de toda a parte visual: menus, listas, botões, vídeos, links, basicamente a estrutura visual do site.

No desenvolvimento web, chamamos esse lado de Front-End, a parte responsável pelos aspectos visuais e interativos da página web, da parte de quem o acessa (cliente).

Esse lado, Front-end envolve o uso das tecnologias como HTML, CSS e JavaScript, (incluindo os seus frameworks e bibliotecas, como ReactJS, VueJs, Angular, JQuery etc). Segundo pesquisas recentes na web, 67, 7% das páginas web são construídas com essas tecnologias supracitadas.

Em outras palavras, são as tecnologias responsáveis por tudo o que o usuário vê e interage em um navegador da web. Isso inclui a estruturação do conteúdo, a estilização visual e a funcionalidade interativa da aplicação. As principais tecnologias do front-end incluem HTML, CSS e JavaScript, além de frameworks e bibliotecas populares que ajudam a facilitar o desenvolvimento e aprimorar a experiência do usuário.

2.4.2.1. REACT

O ReactJS, carinhosamente apelidado de React, é uma biblioteca JavaScript de código aberto criada pelo Facebook e lançada em 2013. Ele se tornou rapidamente uma das ferramentas mais populares para desenvolvimento front-end, sendo utilizado por empresas gigantes como Netflix, Airbnb, Instagram e, claro, o próprio Facebook.

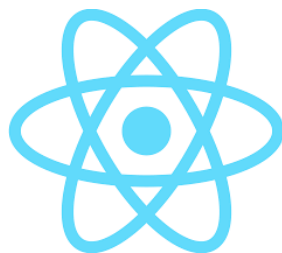


Figura 10 – Logo do React

2.4.2.1.1. PRINCIPAIS CARACTERÍSTICAS DO REACT.JS COMPONENTIZAÇÃO

O React.js promove a construção de interfaces de usuário por meio de componentes, que são pedaços isolados e reutilizáveis de código. Cada componente possui seu próprio estado e comportamento, tornando a UI mais modular e fácil de entender e manter.

VIRTUAL DOM (Documento Objeto Modelo)

O React.js utiliza um Virtual DOM para otimizar a manipulação e atualização eficiente da interface do usuário. O Virtual DOM é uma representação leve e eficiente da árvore de elementos do DOM real, permitindo atualizações rápidas e mínimas na renderização da UI.

JSX (JavaScript XML)

O React.js utiliza o JSX, uma extensão de sintaxe que permite escrever estruturas de UI de forma declarativa, semelhante ao HTML, diretamente no código JavaScript. Isso facilita a criação e a visualização de componentes UI dentro do próprio código JavaScript.

UNIDIRECIONALIDADE DE DADOS (One-Way Data Binding)

O React.js adota o conceito de fluxo unidirecional de dados, o que significa que os dados fluem de cima para baixo na hierarquia de componentes. Isso torna a aplicação mais previsível e fácil de depurar, evitando bugs relacionados a estados inconsistentes.

RENDERIZAÇÃO NO LADO DO CLIENTE E NO SERVIDOR

O React.js suporta a renderização tanto no lado do cliente (client-side rendering) quanto no lado do servidor (server-side rendering), o que permite uma melhor otimização de desempenho e SEO (Search Engine Optimization) para aplicações web.

ECOSSISTEMA ROBUSTO

O React.js possui um ecossistema rico, com uma ampla gama de bibliotecas, ferramentas e frameworks complementares, como React Router para roteamento, Redux para gerenciamento de estado, Material-UI para componentes de interface de usuário, entre outros.

2.4.2.1.2. PRINCIPAIS APLICAÇÕES DO REACT DESENVOLVIMENTO DE APLICATIVOS WEB

O React é amplamente utilizado para desenvolver interfaces de usuário (UI) de alta qualidade para aplicativos web. Com sua abordagem declarativa e componentizada, é ideal para construir interfaces de usuário complexas e interativas.

Single-Page Applications (SPAs)

O React é especialmente popular para criar SPAs, onde todo o conteúdo é carregado em uma única página e as interações do usuário são tratadas dinamicamente, sem a necessidade de recarregar a página. Ele oferece um desempenho excepcional e uma experiência de usuário fluida.

Desenvolvimento de Aplicativos Móveis

Com o React Native, uma extensão do React, é possível desenvolver aplicativos móveis nativos para iOS e Android utilizando JavaScript e React. Ele permite compartilhar grande parte do código entre plataformas, oferecendo uma maneira eficiente de construir aplicativos móveis multiplataforma.

Integração com Outros Frameworks e Bibliotecas

O React é frequentemente usado em conjunto com outros frameworks e bibliotecas, como Redux para gerenciamento de estado, React Router para roteamento, Material-UI para componentes de interface de usuário, entre outros. Ele é altamente extensível e se integra facilmente a outros ecossistemas.

DESENVOLVIMENTO DE APLICATIVOS DE PÁGINA ESTÁTICA (Static Site Generation)

Com o Next.js, um framework baseado em React, é possível criar aplicativos de página estática, onde o conteúdo é pré-renderizado no momento da compilação e servido como HTML estático. Isso melhora o desempenho e a SEO dos sites.

Utilização com o Back-end

A biblioteca pode ser utilizada com qualquer tipo de linguagem back-end, ou seja a aplicação pode ser PHP, Java, Ruby etc.

2.4.2.2. COMPONENTES REACT

Os componentes React são blocos de construção fundamentais em aplicações React. Eles encapsulam a lógica e a interface de usuário de uma parte específica da aplicação em um único pacote. Os componentes podem ser considerados como pequenas partes reutilizáveis e independentes que compõem a interface de usuário de uma aplicação.

Existem dois tipos principais de componentes React:

2.4.2.2.1. COMPONENTES FUNCIONAIS

- ★ São definidos como funções JavaScript.
- ★ Recebem as propriedades (props) como argumento e retornam elementos React que descrevem o que deve ser renderizado na tela.
- ★ São mais simples e geralmente utilizados para componentes sem estado ou lógica de ciclo de vida.
- ★ Com a introdução dos hooks no React, os componentes funcionais podem agora ter estado e usar recursos do ciclo de vida.

2.4.1.2.2. COMPONENTES DE CLASSE

- ★ São definidos como classes JavaScript que estendem a classe React.Component.
- ★ Permitem a definição de estados internos, manipulação de ciclo de vida e gerenciamento de estado.

- ★ São úteis para componentes que precisam de lógica mais complexa ou estado interno.

Observação: Actualmente os componentes de classe não são recomendados para a construção de aplicações React. Segundo a próprio Team do React, é recomendado usar os componentes Funcionais.

Propriedades: (props) são um mecanismo utilizado em React para passar dados de um componente pai para um componente filho. Eles permitem a comunicação entre componentes em uma hierarquia de árvore de componentes React.

Estados: (states) no React são objetos que permitem armazenar dados que podem mudar ao longo do tempo. Eles representam a parte dinâmica de um componente React, que pode ser atualizada e refletida na interface de usuário. Os estados são fundamentais para criar componentes interativos e dinâmicos em aplicações React

Ciclo de vida dos componentes: O ciclo de vida no React refere-se ao ciclo de vida de um componente React, ou seja, às fases pelas quais um componente passa desde sua inicialização até sua remoção do DOM.

Figura 13 - Ciclo de vida de um componente no react

2.4.2.3. NextJS

Next.js é uma estrutura da web de desenvolvimento front-end React de código aberto criada por Vercel que permite funcionalidades como renderização do lado do servidor e geração de sites estáticos para aplicativos da web baseados em React. É uma estrutura pronta para produção que permite que os desenvolvedores criem rapidamente sites JAMstack estáticos e dinâmicos e é amplamente usado por muitas grandes empresas.

Ele oferece uma experiência de desenvolvimento intuitiva e eficiente, fornecendo uma série de recursos e funcionalidades que agilizam o processo de criação de aplicações web.

2.4.2.3.1. Principais características do Next.js

Renderização Híbrida: Next.js suporta tanto a renderização no lado do servidor (Server-Side Rendering - SSR) quanto a renderização no lado do cliente (Client-Side Rendering - CSR).

Isso permite que os desenvolvedores escolham a abordagem mais adequada para suas necessidades de projeto.

Pré-processamento de Páginas: As páginas em Next.js são pré-processadas no momento da compilação, o que resulta em tempos de carregamento mais rápidos e melhor desempenho. Isso é especialmente útil para otimização de SEO (Search Engine Optimization) e experiência do usuário.

Rotas Dinâmicas: Next.js permite a criação de rotas dinâmicas, onde os parâmetros da URL podem ser usados para gerar conteúdo de forma dinâmica. Isso simplifica a criação de páginas dinâmicas e flexíveis em aplicações web.

Gerenciamento de Estado: Next.js oferece suporte para diferentes abordagens de gerenciamento de estado, como o uso de React Context API, Redux, Zustand ou gerenciamento de estado local com hooks do React.

API Routes Handlers: Permite a criação de APIs RESTful diretamente no próprio aplicativo Next.js, facilitando a implementação de backends simples e integrados com o frontend.

Back-end from Front-end: Essa é uma nova estratégia que esse framework do React utiliza para poder criar aplicações completas (Full-Stacks). Com a sua integração com o Prisma, Vercel Postgres, etc, criar aplicações completas fica muito fácil.

2.4.2.4. Tailwind

Tailwind CSS é uma estrutura CSS de código aberto . A principal característica desta biblioteca é que, diferentemente de outros frameworks CSS como Bootstrap , ela não fornece uma série de classes predefinidas para elementos como botões ou tabelas. Em vez disso, ele cria uma lista de classes CSS "utilitárias" que podem ser usadas para estilizar cada elemento misturando e combinando.



Figura 11 – Logo do TailwindCSS

É importante referir que o desenvolvimento Front-end envolve mais do que essas tecnologias citadas acima, tempo nos faltaria se citássemos todas elas, porque são muitas, muitas mesmo. Aqui nós focamos apenas nas tecnologias usadas no âmbito do desenvolvimento deste projecto. Mas ainda existem muitas outras por aí como: Styled Components, Chakra UI, Material UI, VueJs, Angular, Svelte, Solid, Remix, Astro, PostCss, Saas, e muitas etc.

2.4.4. TECNOLOGIAS DO BACK-END

O backend, também conhecido como servidor ou parte do servidor, refere-se à parte de um sistema de software que lida com a lógica de processamento, armazenamento e recuperação de dados. Em aplicações web, o backend é responsável por atender às solicitações dos clientes (geralmente navegadores da web), processar essas solicitações e retornar as respostas apropriadas.

O backend contém a lógica de negócios da aplicação, o que significa que ele executa as operações e cálculos necessários para atender às necessidades específicas do negócio ou do aplicativo.

Desenvolvedores backend focam principalmente em como o website funciona. Eles escrevem códigos focados na funcionalidade e na lógica que faz o aplicativo web que eles trabalham funcionar, e a tecnologia que eles utilizam nunca é vista pelos usuários que acessam a aplicação web. As tecnologias backend são uma combinação de servidores, aplicações e banco de dados.

As responsabilidades de um desenvolvedor backend podem envolver no desenvolvimento de APIs, escrever códigos para interagirem com o banco de dados, criar bibliotecas, trabalhar em processos dos negócios, estrutura de dados e muito mais. No que o desenvolvedor backend vai trabalhar dependem do cargo que ele vai preencher e a empresa.

De acordo com o desenvolvedor backend Ali Bayat em um artigo de seu blog *The Day in the Life of a Backend Developer*, “Refletindo sobre o dia típico de trabalho, eu diria que passo 50% do tempo em trabalho de projeto, 20% em reuniões, comunicações em geral e os 30% eu trabalho em pedidos imediatos e bugs”.

Então, como desenvolvedor backend, você talvez passe o seu dia fazendo coisas como:

- ★ Mantendo aplicações legacy (programas antigos que foram construídos por outros programadores);
- ★ Apagando o fogo (por exemplo, bugs, aplicativos quebrados);
- ★ Indo em reuniões (para conseguir exigências para um projeto);
- ★ Colaborar com desenvolvedores front end em um projeto, etc.
- ★ De fato escrever códigos para novas aplicações/projetos

Então, enquanto você certamente estará programando com linguagens back end, não pense que isso será a única coisa que irá fazer como um desenvolvedor back end.

2.4.4.1. TIPOS DE LINGUAGENS BACKEND

A programação em backend pode tanto orientada a objetos/object-oriented (OOP) ou funcional.

POO é a técnica que foca na criação de objetos. Com a programação orientada a objetos, os códigos precisam ser executados de uma forma em particular. Linguagens POO populares são Java, .NET e Python.

Programação back end funcional é uma técnica mais focada na “ação”. A programação funcional utiliza a linguagem declarativa, isso significa que os statements podem ser executados de qualquer ordem. É comumente utilizado no data science e em linguagens populares como SQL, F# e R.

As linguagens back-end podem ser escritas tanto de forma estática ou dinâmica. O primeiro é mais rígido, mas melhor para encontrar erros, onde que o último é mais flexível, mas permite com que as variáveis mudem os tipos (no qual pode contar com erros inesperados).

2.4.4.2. NodeJS

Node.js é uma plataforma de desenvolvimento de software de código aberto, baseada no motor JavaScript V8 do Google Chrome, que permite a execução de código JavaScript no lado do servidor. Ele fornece um ambiente de tempo de execução assíncrono e orientado a eventos, projetado para criar aplicativos escaláveis e de alto desempenho.

2.4.4.2.1. CARACTERÍSTICAS DO NodeJS

JavaScript no Servidor: Node.js permite que os desenvolvedores usem JavaScript tanto no lado do cliente quanto no lado do servidor, unificando o ambiente de desenvolvimento e permitindo a reutilização de código em ambas as partes da aplicação.

Modelo de E/S Não-Bloqueante: Node.js utiliza um modelo de E/S não- bloqueante, o que significa que ele pode lidar com múltiplas operações de entrada e saída de forma assíncrona, sem bloquear a execução do programa. Isso torna Node.js ideal para aplicações que requerem alta concorrência e escalabilidade.

Módulos Nativos e Gerenciador de Pacotes: Node.js possui um sistema de módulos nativos e um gerenciador de pacotes chamado npm (Node Package Manager), que oferece acesso a uma vasta biblioteca de pacotes e módulos de terceiros para ampliar a funcionalidade do Node.js e acelerar o desenvolvimento de aplicativos.

Desenvolvimento Rápido: Node.js é conhecido por seu ambiente de desenvolvimento rápido e eficiente, com ferramentas poderosas e uma comunidade ativa que oferece suporte e recursos adicionais para os desenvolvedores.

Suporte a APIs Web: Node.js é amplamente utilizado para criar APIs web RESTful e serviços web, devido à sua capacidade de lidar com solicitações HTTP de forma eficiente e escalável.

Streaming de Dados: Node.js oferece suporte nativo para streaming de dados, o que permite processar grandes volumes de dados de forma eficiente e em tempo real, tornando-o ideal para aplicações de transmissão de mídia, análise de dados e outras aplicações que lidam com grandes quantidades de dados.

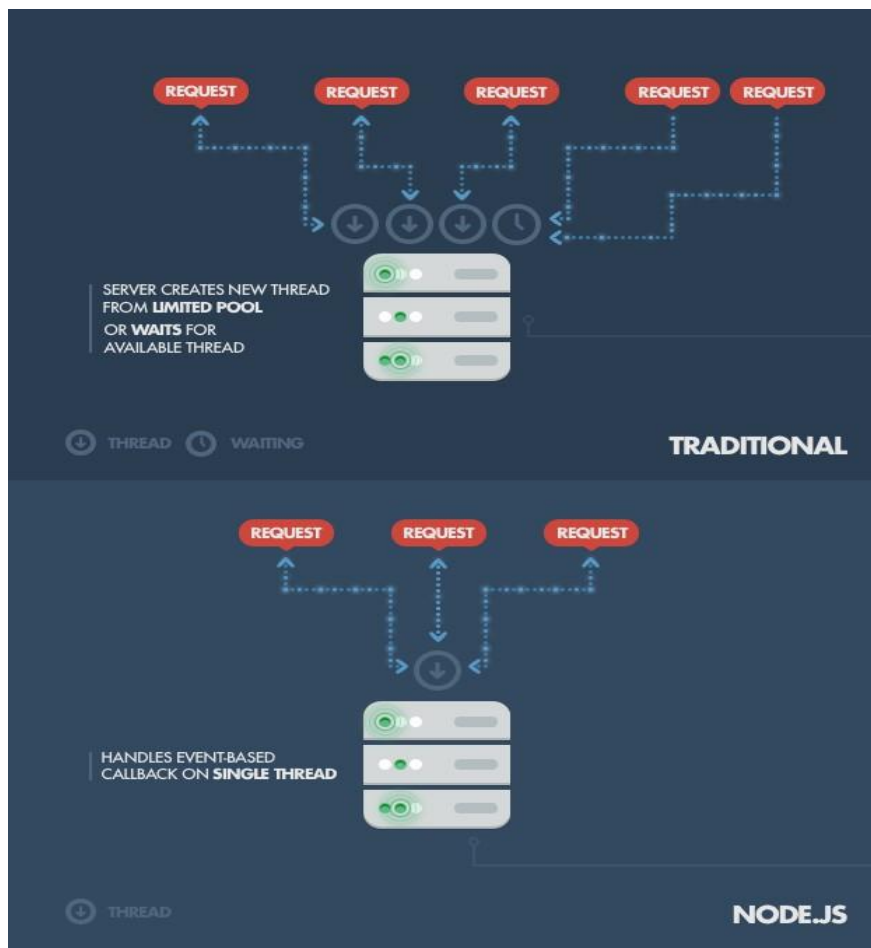


Figura 12 – Processo de requisições de um servidor NodeJS

2.4.4.2.2. Uso do NodeJs

Com Node.js, é possível criar diversos tipos de aplicações, como: Aplicações de chat em tempo real, APIs, Micro Serviços e um gama de outras aplicações, usando como linguagem base o JavaScript.

2.4.4.2.2.1. PROXY

Um proxy em Node.js é uma aplicação que atua como intermediário entre o cliente e o servidor. Ele recebe solicitações de clientes, as encaminha para o servidor de destino e retorna as respostas ao cliente. Os proxies são frequentemente usados para roteamento de tráfego, balanceamento de carga, ocultação de origem e muito mais.

2.4.4.2.3. EXPRESS

Express.js (ou apenas Express) é um framework para Node.js que fornece recursos mínimos para construção de servidores web (HTTP). Foi lançado como software livre e de código aberto sob a Licença MIT. É um dos mais populares frameworks para servidores em Node.js.

2.4.4.2.4. VANTAGENS DE USAR DO NodeJs

2.4.4.2.4.1. FLEXIBILIDADE

O Node.js possui o gerenciador com inúmeros pacotes e software reusável NPM (Node Package Manager), dando ao interpretador um potencial a ser utilizado em qualquer situação. O pacote mais conhecido é o Express.js, um arcabouço (framework) completo para desenvolvimento de aplicações web.

2.4.4.2.4.2. LEVEZA

Um ambiente Node.js não exige muitos recursos computacionais tradicionais. Se utilizado em conjunto com a ferramenta Docker, o ganho na velocidade de deployment e replicação de máquinas pode ser muito significativo e, em ambientes escaláveis, de microserviços e serverless, representa menos custo e mais eficiência. Um exemplo de uso é uma aplicação de troca de mensagens (conversa, bate-papo ou webchat).

2.4.4.2.4.3. SUPORTE

Conta com suporte das principais empresas de serviços de armazenamento na nuvem, como a AWS, Google Cloud e Microsoft Azure, com suporte nativo ao Node.JS. 2.5.3.2.4.4. Produtividade

O gerenciador NPM fornece pacotes reusáveis disponíveis gratuitamente para uso em grupo.

Mesma linguagem no front end e back end, que pode representar ganhos de reúso de código e criação de equipes multidisciplinares, reaproveitando recursos;

A possibilidade de deployments e iterações mais rápidas, e resolução de problemas on the fly, também permitem a criação de soluções próprias e inovadoras, como a Uber faz para resolver alguns problemas.



CAPITULO III

BANCO DE DADOS

3. BANCO DE DADOS

Bancos de dados ou bases de dados são conjuntos de arquivos relacionados entre si, podendo conter registros sobre pessoas, lugares ou informações em geral. São coleções organizadas de dados que se relacionam ou não, com o objetivo de armazenar informações. São de vital importância para empresas e tornaram-se a principal fonte de dados dos sistemas de informação e segurança. Normalmente, eles existem por vários anos sem alterações em sua estrutura sistemática.

A principal aplicação de banco de dados é o controle de operações empresariais, normalmente armazenando dados pertinentes para a administração de um negócio, como dados de clientes, funcionários, fornecedores e outras informações. Outra aplicação importante é o gerenciamento de informações de estudos, como fazem os Bancos de Dados Geográficos, que unem informações como mapas, imagens de satélite, pontos, linhas, áreas, entre outros dados geográficos

3.1. TIPOS DE BANCO DE DADOS

Existem vários tipos de banco de dados, cada um projetado para atender a diferentes necessidades e cenários de uso. Aqui estão alguns dos tipos mais comuns de banco de dados:

3.1.1. BANCOS DE DADOS RELACIONAIS (RDBMS):

Exemplos: MySQL, PostgreSQL, Oracle, SQL Server.

Armazenam dados em tabelas relacionadas usando um esquema predefinido.

Utilizam a linguagem SQL (Structured Query Language) para consultar e manipular dados.

Oferecem transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade) para garantir a integridade dos dados.

3.1.2. BANCOS DE DADOS NÃO-RELACIONAIS (NoSQL):

Exemplos: MongoDB, Cassandra, Redis, Couchbase.

Utilizados para armazenar dados não estruturados ou semiestruturados.

Não possuem esquema fixo, permitindo flexibilidade na modelagem de dados.

Podem ser otimizados para escalabilidade horizontal e manipulação de grandes volumes de dados.

3.1.3. BANCOS DE DADOS DE GRAFOS

Exemplos: Neo4j, Amazon Neptune, ArangoDB.

Projetados para armazenar e consultar dados relacionados, onde os relacionamentos são tão importantes quanto os próprios dados.

Utilizam grafos para representar dados como nós (entidades) e arestas (relacionamentos) entre eles.

São eficientes para consultas complexas que envolvem relacionamentos entre entidades.

3.1.4. BANCOS DE DADOS EM MEMÓRIA

Exemplos: Redis, Memcached.

Armazenam dados na memória principal do servidor para acesso rápido.

São utilizados para armazenamento de cache, sessões de usuário, filas de mensagens e outras aplicações que requerem acesso rápido aos dados.

3.2. SGBD

Os bancos de dados são operados por Sistemas Gerenciadores de Bancos de Dados (SGBD), que surgiram na década de 1970. Antes destes, as aplicações usavam sistemas de arquivos do sistema operacional para armazenar suas informações.

Na década de 1980, a tecnologia de SGBD relacional passou a dominar o mercado; atualmente, é utilizada em praticamente todos os bancos de dados.

Outro tipo notável é o SGBD orientado a objetos, implementado em bancos de dados com estruturas complexas ou aplicações que mudam constantemente.[4] Alguns dos SGBD mais usados atualmente incluem o MySQL, Oracle, PostgreSQL, Microsoft SQL Server, SQLite entre outros sistemas que gerenciam bases de dados.

3.3. VANTAGENS DE USAR BANCO DE DADOS

Usar um banco de dados traz uma série de vantagens para o desenvolvimento de software e para o armazenamento de dados. Aqui estão algumas das principais vantagens:

Organização e Estruturação dos Dados: Os bancos de dados permitem organizar os dados de forma estruturada em tabelas (no caso de bancos de dados relacionais) ou em formatos mais flexíveis (no caso de bancos de dados NoSQL), facilitando a consulta e a manipulação dos dados.

Integridade dos Dados: Os bancos de dados garantem a integridade dos dados, implementando restrições e validações para garantir que apenas dados válidos sejam armazenados. Isso ajuda a manter a consistência e a precisão dos dados ao longo do tempo.

Recuperação Eficiente de Dados: Os bancos de dados oferecem mecanismos eficientes para recuperar dados por meio de consultas, índices e otimização de consultas. Isso permite que os aplicativos recuperem os dados necessários de forma rápida e eficiente, mesmo em conjuntos de dados grandes.

Escalabilidade: Os bancos de dados são projetados para escalar horizontalmente e verticalmente, permitindo lidar com grandes volumes de dados e alto tráfego de maneira eficiente. Isso é essencial para aplicativos que precisam crescer e se adaptar a demandas crescentes.

Segurança dos Dados: Os bancos de dados oferecem recursos de segurança robustos, como controle de acesso, criptografia, auditoria e backups. Isso ajuda a proteger os dados contra acessos não autorizados, perda de dados e corrupção.

Backup e Recuperação: Os bancos de dados permitem fazer backups regulares dos dados e oferecem recursos de recuperação em caso de falhas ou desastres. Isso ajuda a garantir a disponibilidade e a durabilidade dos dados ao longo do tempo.

Essas são apenas algumas das grandes vantagens de usar um banco de dados numa aplicação web, mobile ou desktop.

3.4. PHP

PHP (Hypertext Preprocessor) é uma linguagem de programação amplamente utilizada no desenvolvimento web, especialmente para aplicações dinâmicas que interagem com bancos de dados. Graças à sua simplicidade, ampla documentação e integração com diversos SGBDs (como MySQL e PostgreSQL), PHP permanece uma das escolhas mais populares para aplicações web no lado do servidor.

3.4.1. CARACTERÍSTICAS

Eis as principais características do PHP

- ★ Código executado no servidor;
- ★ Ampla integração com bancos de dados;
- ★ Sintaxe simples e flexível;
- ★ Grande comunidade e vasta documentação;
- ★ Compatibilidade com servidores web;
- ★ Suporte a programação orientada a objetos (POO);
- ★ Segurança;

3.5. ORMs

3.5.1. Eloquent ORM

O **Eloquent ORM** é a biblioteca de mapeamento objeto-relacional utilizada pelo framework **Laravel**, um dos mais populares em PHP. Ele permite interações simples e elegantes com o banco de dados usando sintaxe orientada a objetos.

As principais características e benefícios do Eloquent ORM são:

Mapeamento de tabelas em classes PHP

Cada modelo representa uma tabela do banco de dados.

Consultas fluentes

Consultas escritas com métodos encadeados, fáceis de ler e manter.

Sintaxe de Consulta Simples:

Em vez de escrever consultas SQL diretamente, os desenvolvedores podem usar uma sintaxe de consulta mais amigável, baseada em métodos encadeados e operações com objetos.

Relacionamentos entre Modelos

Suporte a relacionamentos `hasOne`, `hasMany`, `belongsTo`, `belongsToMany`.

3.6. Doctrine ORM

O **Doctrine ORM** (Object-Relational Mapper) é uma poderosa biblioteca de mapeamento objeto-relacional para **PHP**, amplamente utilizada no desenvolvimento de aplicações robustas e escaláveis. Criado com foco em abstração e boas práticas, o Doctrine permite que desenvolvedores interajam com bancos de dados relacionais por meio de classes e objetos PHP, eliminando a necessidade de escrever SQL manualmente na maioria dos casos.

Muito utilizado em conjunto com o framework **Symfony**, mas também funcional de forma independente, o Doctrine se destaca por sua **flexibilidade**, **modularidade** e **suporte a padrões de projeto** que favorecem a organização do código e a manutenção a longo prazo.

São elas algumas características principais do Doctrine ORM:

Independência de framework: O Doctrine ORM é uma biblioteca independente, o que significa que ele não está amarrado a nenhum framework específico. Isso permite que seja utilizado em projetos personalizados ou integrado a qualquer framework PHP, como Laravel, Symfony, Zend, Slim, entre outros.

Anotações ou arquivos YAML/XML: O Doctrine oferece flexibilidade na definição dos modelos e metadados das entidades (ou seja, as estruturas que representam as tabelas no banco de dados). Essa variedade de métodos permite que os desenvolvedores escolham o estilo que melhor se adapta à equipe ou ao projeto, promovendo clareza e organização no mapeamento entre os objetos PHP e as tabelas relacionais.

Geração automática de esquema Uma das grandes vantagens do Doctrine é sua capacidade de gerar e sincronizar automaticamente o esquema do banco de dados com base nas entidades PHP definidas no projeto.

QueryBuilder: O QueryBuilder do Doctrine é uma ferramenta poderosa que permite criar consultas SQL de forma programática e segura, usando uma interface orientada a objetos.



CAPITULO IV

METODOLOGIA DE DESENVOLVIMENTO

4. METODOLOGIAS DE DESENVOLVIMENTO

4.1. Prontuário Eletrônico

Como foi referido antes, o Prontuário Eletrônico é um sistema digital utilizado para armazenar e gerenciar informações médicas dos pacientes. Com isso, os profissionais da saúde conseguem acessar e registrar os dados clínicos de maneira rápida e segura. Todo Prontuário Eletrônico possui uma estrutura organizada, que delimita onde as informações começam e onde devem ser armazenadas. Dentro dessa estrutura, existem seções específicas para dados pessoais, histórico médico, prescrições e exames, que ficam sempre em destaque. São elas as responsáveis pela organização do prontuário e também por garantir sua funcionalidade.

Por isso, sabemos que é ali que se registram diagnósticos, tratamentos e outras informações relevantes. Nessas seções também se define a navegação dentro do sistema, permitindo que os profissionais encontrem rapidamente os dados necessários, independentemente da unidade de saúde onde estejam acessando.



Figura 13 - Imagem do Prontuário Eletrônico

4.1.1. USO E TIPO DE PRONTUÁRIOS

O Prontuário Eletrônico pode ser usado para diversas finalidades no dia a dia da área da saúde. Em Angola, por exemplo, ele é utilizado por diversos motivos, como o registro e acompanhamento do histórico médico dos pacientes, a prescrição eletrônica de medicamentos, a marcação de consultas e a integração entre diferentes unidades de saúde, garantindo um atendimento mais eficiente e seguro.

Existem dois tipos de Prontuários Eletrônicos: **Local** e em **Nuvem**.

Os **prontuários em nuvem utilizam** um sistema online que permite o acesso remoto aos dados do paciente. Este tipo de prontuário oferece muitas vantagens, como a possibilidade de acessar informações em diferentes dispositivos, facilitar a integração entre unidades de saúde e garantir maior segurança com backups automáticos. Além disso, permite o registro de estatísticas sobre atendimentos, histórico de consultas e atualizações médicas em tempo real

Os **prontuários locais** são armazenados diretamente em um computador ou servidor interno da unidade de saúde. Eles não podem ser acessados remotamente e não possuem integração automática com outros sistemas médicos, sendo necessários backups manuais para evitar a perda de dados.

4.2. UML

UML (Unified Modelling Language) “*é a linguagem que utiliza uma notação padrão para especificar, construir, visualizar e documentar sistemas para informação orientados por objetos*”. (NUNES e O'NEILL, 2002)

Pela abrangência e simplicidade dos conceitos utilizados, a UML facilita o desenvolvimento de um sistema de informação. Permite integrar os aspectos de natureza organizacional que constituem o negócio e os elementos de natureza tecnológica, que irão constituir o sistema informático, ajudando a dominar a complexidade das regras de negócio e definir os processos e fluxos informativos. (NUNES e O'NEILL, 2002)

4.3 METODOLOGIAS ÁGEIS

A maioria do desenvolvimento de software é feita de forma caótica, vulgarmente caracterizada pelo método *programar e corrigir*. O código é feito sem um planeamento aprofundado do problema, sendo muitas vezes agregado a porções de código distinto para a resolução de pequenos problemas que surgem conforme o software vai sendo desenvolvido. (TEIXEIRA, PIRES, *et al.*, 2015)

Este procedimento é *eficaz para pequenas aplicações* mas, com o aumento do sistema construído, torna-se bastante mais difícil adicionar novas funcionalidades ao programa ou alterar outras já existentes. Além disso, os *bugs* tornam-se bastante mais frequentes e bem mais difíceis de resolver. (TEIXEIRA, PIRES, *et al.*, 2015)

Uma longa fase de testes posterior à conceção do programa é uma característica deste tipo de programação, bem como um sinal de que foi usada. Dado ser muito difícil estabelecer prazos exatos para uma fase de testes e *debugging* tão longa, o tempo previsto para a execução do trabalho é também difícil de determinar. (TEIXEIRA, PIRES, *et al.*, 2015)

Este estilo de desenvolvimento de projetos foi predominante durante muito tempo, tendo como mais forte concorrente o *Desenvolvimento Metódico*. O *Desenvolvimento Metódico* impõe um processo de desenvolvimento disciplinado, com o objetivo de tornar o desenvolvimento de software previsível e eficiente, conseguido através do desenvolvimento de documentação detalhada e dando um grande ênfase ao planeamento inspirado noutras áreas da engenharia de software. (TEIXEIRA, PIRES, *et al.*, 2015)

O desenvolvimento metódico já existe há bastante tempo, mas nunca chegou a ser frequentemente utilizado devido à sua falta de eficácia – é uma metodologia burocrática sendo que, para a seguir, é necessário cumprir diversos itens que tornam o desenvolvimento do software muito mais demorado. (TEIXEIRA, PIRES, *et al.*, 2015)

4.3.1. O DSDM

DSDM, “*é uma Metodologia Ágil de desenvolvimento de software cujo objetivo é entregar softwares no tempo e com custo estimados através do controle e ajuste de requisitos ao longo do desenvolvimento.*” (TEIXEIRA, PIRES, *et al.*, 2015)

Como uma extensão do RAD (Desenvolvimento Rápido de Aplicação), o DSDM é aplicado em projetos de Sistemas caracterizados pelos cronogramas e custos limitados. Aponta *falhas* de informação mais comuns destes projetos, incluindo custos excedentes, perda de prazos, falta de envolvimento de usuários e acompanhamento da alta gerência. Através do uso do RAD, contudo, sem os devidos cuidados com o DSDM pode aumentar ainda mais o risco em outros quesitos. (TEIXEIRA, PIRES, *et al.*, 2015)

4.3.1.1. PRINCÍPIOS DA DSMD

A DSDM segue alguns princípios chave. Estes princípios delimitam as bases do desenvolvimento utilizando DSDM.

- ★ O ponto fundamental desta metodologia prende-se com a entrega de um sistema que se aproxime das atuais necessidades de negócio. Não é uma metodologia tão direta que forneça todas as necessidades de negócio, mas centraliza todo o potencial na concretização final de todos os objetivos do projeto.

- ★ Nenhum sistema é completamente construído na primeira tentativa. Num processo de desenvolvimento de um sistema informático 80% da solução pode ser desenvolvida em 20% do tempo necessário para encontrar a solução perfeita. Para aperfeiçoar a parte final poderá ser necessário que o projeto ultrapasse o seu tempo e orçamento estipulados. Uma vez que a DSDM é caracterizada por realizar exatamente o que a empresa necessita, é muitas vezes desnecessário chegar à solução perfeita.
- ★ A entrega do projeto deve ser feita na data estipulada, dentro do orçamento previsto e com boa qualidade (Fig. 2).
- ★ As exigências para o Sistema de Informação têm que ser flexíveis. Tal como falaremos mais tarde, exigências flexíveis são tópicos importantes da DSDM.
- ★ Esta metodologia apenas requer que cada etapa do desenvolvimento seja completada até que seja possível iniciar o passo seguinte. Isto faz com que cada fase do projeto possa começar sem ter que esperar que as fases que começaram anteriormente sejam totalmente terminadas.
- ★ A comunicação entre todas as partes envolvidas (stakeholders) é também um pré-requisito bastante importante para que o projeto corra com a eficiência desejada.
- ★ O envolvimento dos utilizadores é a chave para esta eficiência.
- ★ As equipas responsáveis têm que ser dotadas de um sentido de decisão, sendo este também um ponto fulcral na progressão do projeto.
- ★ Tal como as equipas de desenvolvimento também as equipas de gestão do projeto estão incorporadas na DSDM.

Após o desenvolvimento do Sistema de Informação, a DSDM pode também ser usado para expandir o Sistema obtido.

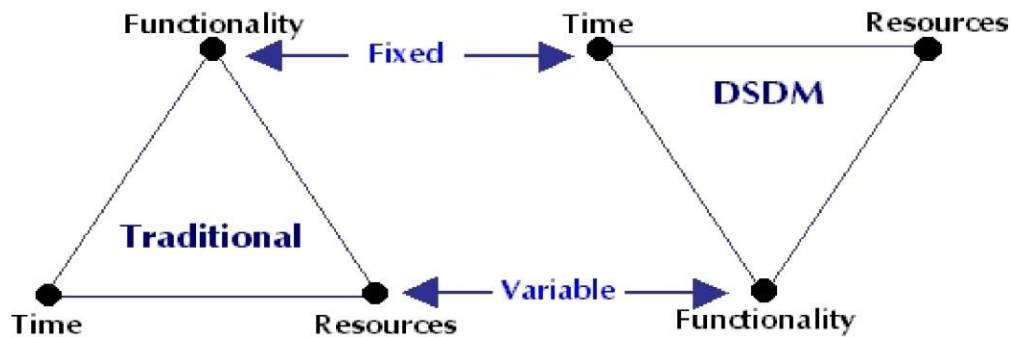


Figura 14 - Princípio fundamental da DSDM

4.3.1.2. PRÉ-REQUISITOS PARA A UTILIZAÇÃO DA DSDM

Para que a DSDM seja um sucesso, um número definido de pré-requisitos devem ser implementados.

- ★ Em primeiro lugar, é necessária a existência de uma interatividade entre a equipa de desenvolvimento, os utilizadores finais e os gestores de projeto. A falta de motivação da equipa de gestão ou a falta de envolvimento dos utilizadores finais é uma das mais frequentes causas de falha no desenvolvimento de projetos de Sistemas de Informação.
- ★ Um segundo, e importante, pré-requisito é a facilidade de decomposição em módulos. Esta possibilidade de decompor o projeto em partes mais pequenas, possibilita as abordagens iterativa e incremental da DSDM. Esta funcionalidade possibilita a criação de diversos projetos mais pequenos, também eles a serem desenvolvidos segundo os princípios da DSDM.
- ★ A terceira, e não menos importante, propriedade relaciona-se com a possibilidade de definir claramente os requisitos do SI, introduzindo-lhes uma ordem segundo a sua prioridade de implementação.

4.3.1.3. AS FASES DA DSDM

O *framework* DSDM consiste em três fases sequenciais: **Pré-projecto**, **Projeto** e **Pós-projecto**. A fase de Projeto do DSDM é a mais elaborada das três fases (TEIXEIRA, PIRES, *et al.*, 2015). Ela consiste em 5 níveis formados por uma abordagem passo-a-passo e iterativa no desenvolvimento de um SI. As três fases e correspondentes níveis são explicadas exhaustivamente nas secções seguintes:

Fase 1: O Pré-projecto

Na fase do pré-projecto, o projeto candidato é identificado, tratando-se depois do seu plano de financiamento e sendo assegurado um compromisso de realização. Tratar destas questões numa fase inicial evita problemas futuros em fases mais avançadas do desenvolvimento do projeto. (TEIXEIRA, PIRES, *et al.*, 2015)

Fase 2: O Ciclo de Vida do Projeto

A visão geral de um processo DSDM, presente na Fig. 3, representa o **Ciclo de Vida** do Projeto nesta segunda fase da metodologia. Ela mostra os 5 níveis que a equipa de desenvolvimento terá de percorrer para criar um SI. Os dois primeiros níveis, o **Estudo de Viabilidade** e o **Estudo de Negócio**, são fases sequenciais que se complementam. (TEIXEIRA, PIRES, *et al.*, 2015)

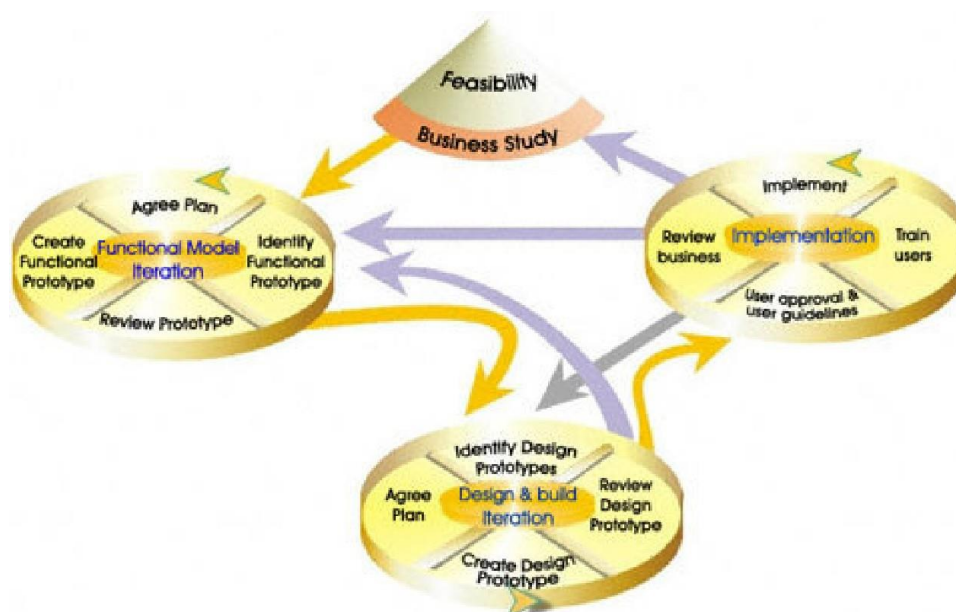


Figura 15 - Ciclo de Vida de um Projeto em DSDM

Depois destas fases estarem concluídas, o sistema é desenvolvido iterativamente e de forma incremental nos níveis de Análise Funcional, Desenho e Implementação.

Nível 1: O Estudo de Viabilidade

Durante este nível do projeto, a viabilidade de utilização da DSDM para este projeto é examinada. Os pré-requisitos para o uso da DSDM são encontrados respondendo a questões como:

“Pode este projeto ir de encontro às necessidades de negócio apontadas?”, “É, este projeto, adequado ao uso da DSDM?” e “Quais são os riscos mais importantes envolvidos?”. As técnicas mais importantes utilizadas nesta fase são os *workshops*. (TEIXEIRA, PIRES, *et al.*, 2015).

Para entrega ao cliente, são preparados neste nível o **Relatório** e o **Protótipo de Viabilidade** que dizem respeito à viabilidade do projeto em mãos. A estes, adicionam-se um esboço global do plano para o resto do projeto e um **Registo de Risco** que identifica os riscos mais importantes no projeto. (TEIXEIRA, PIRES, *et al.*, 2015)

Nível 2: O Estudo do Negócio

O Estudo do Negócio incrementa todo o trabalho realizado no Estudo de Viabilidade. Depois do projeto ser declarado fiável para o uso da DSDM, este nível examina o processo de financiamento, os utilizadores envolvidos e as suas necessidades e desejos respetivos. Uma vez mais, os *workshops* são uma das mais valiosas técnicas. *Workshops* nos quais os diferentes *stakeholders* se reúnam e discutam o sistema proposto. (TEIXEIRA, PIRES, *et al.*, 2015)

A informação retirada destas sessões é combinada numa lista de requisitos. Uma importante propriedade desta lista de requisitos é a possibilidade de definir prioridades. Estas prioridades são definidas utilizando uma perspetiva **MoScow1**. Baseado neste escalonamento, um plano de desenvolvimento é construído como uma linha mestra para o resto do projeto. Uma importante técnica utilizada no desenvolvimento do plano é a técnica de **Timeboxing2**. Esta técnica é essencial para serem atingidos os objetivos da DSDM, nomeadamente a imposição de tempo e orçamento fixos, garantindo no entanto a qualidade desejada. Uma arquitetura de sistema é outro meio para guiar o desenvolvimento do Sistema de Informação. (TEIXEIRA, PIRES, *et al.*, 2015).

No final deste nível, deverão estar prontos para entrega ao cliente: uma definição de área de negócio que descreve o contexto do projeto dentro da companhia, a definição da arquitetura do sistema que fornece uma arquitetura global inicial do SI em desenvolvimento juntamente com o plano de desenvolvimento que realça os passos mais importantes no processo de desenvolvimento. Na base destes dois últimos documentos está a lista de prioridades dos requisitos. A lista define todos os requisitos do sistema, organizados de acordo com o princípio do *MoSCoW*. Por fim, o Registo de Risco é atualizado com os factos que foram identificados durante esta fase da DSDM. (TEIXEIRA, PIRES, *et al.*, 2015)

Nível 3: Análise Funcional

Os requisitos que foram identificados nos níveis anteriores são convertidos para um **Modelo Funcional**. A **Prototipagem** é uma das técnicas chave dentro deste nível, que ajuda no bom envolvimento do utilizador final com o projeto (TEIXEIRA, PIRES, *et al.*, 2015). O protótipo desenvolvido é revisto pelos diferentes grupos de utilizadores. Para assegurar a qualidade do projeto, os testes são implementados em cada iteração da DSDM. Uma importante parte dos testes são realizados na Análise Funcional (TEIXEIRA, PIRES, *et al.*, 2015). O Modelo Funcional pode ser subdividido em quatro subníveis:

Identificar Protótipo Funcional: determinar as funcionalidades a ser implementadas no protótipo que resulta desta iteração.

Acordar Calendário de Tarefas: definir como e quando desenvolver estas funcionalidades.

Criar Protótipo Funcional: desenvolver o protótipo.

Rever o Protótipo: procurar correções possíveis no protótipo desenvolvido. Isto pode ser feito através de testes com utilizadores finais e revendo a documentação.

Neste nível, é necessário entregar ao cliente o Modelo Funcional e o Protótipo Funcional que, juntos, representam as funcionalidades que podem ser realizadas nesta iteração, prontas para serem testadas pelos utilizadores. Além destes dois documentos, a Lista de Requisições é atualizada, sendo apagados os itens que foram implementados e repensando as prioridades dos requisitos restantes. (TEIXEIRA, PIRES, *et al.*, 2015)

Nível 4: Desenho

O ponto central desta iteração da DSDM é a integração das componentes funcionais do nível anterior num sistema que satisfaça as necessidades do utilizador. Mais uma vez, os testes são uma das atividades mais importantes (TEIXEIRA, PIRES, *et al.*, 2015). O Desenho pode ser dividido em quatro subníveis:

Identificar Protótipo de Desenho: identificar requisições funcionais e não-funcionais que são necessárias no sistema testado.

Acordar Calendário de Tarefas: definir como e quando desenvolver estes requisitos.

Criar Protótipo de Desenho: criar um sistema que possa, com segurança, ser fornecido aos utilizadores finais para um uso diário.

Rever Protótipo de Desenho: verificar a exatidão do sistema desenhado. Mais uma vez, os testes e revisões são peças fundamentais.

Ao utilizador, será entregue o Protótipo de Desenho para que estes efetuem testes ao produto-protótipo.

Nível 5: Implementação

No nível de Implementação, o sistema testado e a sua documentação são entregues aos utilizadores finais que deverão começar a ser treinados para a futura utilização do novo SI. O sistema a ser entregue foi revisto para incluir todos os requisitos que foram definidos nos primeiros níveis do projeto. O nível de implementação pode ser dividido em quatro subníveis:

Aprovação do utilizador: os utilizadores finais aprovam o sistema testado para implementação e as linhas mestras para a implementação e uso do sistema são criadas.

Treinar os utilizadores: treinar os futuros utilizadores no uso do sistema.

Implementação: implementar o sistema testado no local de trabalho dos utilizadores finais.

Rever Negócio: rever o impacto do sistema implementado no negócio, um problema central será tentar compreender se o sistema vai de encontro aos objetivos definidos no início do projeto. Dependendo disto, o projeto passará para a fase seguinte, o Pós-Projeto ou voltará a uma das fases anteriores para desenvolvimento posterior. (TEIXEIRA, PIRES, *et al.*, 2015)

No final deste nível, o sistema deverá ser entregue e instalado, pronto para o uso de todos os utilizadores finais e a **Documentação de Utilização do Sistema** deverá ser detalhada.

Fase 3: Pós-Projeto

A fase de pós-projeto assegura um sistema de atuação eficiente. Isto é implementado através da manutenção e melhoramentos de acordo com os princípios da DSDM. Até mesmo a iniciação de novos projetos, para atualizar o sistema existente ou desenvolver um novo sistema, é possível. (TEIXEIRA, PIRES, *et al.*, 2015)



CAPITULO V

**ANÁLISE E DESENHO DO SISTEMA E
TECNOLOGIAS USADAS**

5 – ANÁLISE E DESENHO DO SISTEMA E TECNOLOGIAS USADAS

5.1. VISUALIZAÇÃO DO PRONTUÁRIO DO PACIENTE

Como parte integrante do sistema desenvolvido para a secretaria do Instituto Politécnico Industrial de Luanda (IPIL), foi criada uma aplicação mobile que visa facilitar o processo de pagamento do RUP pelos alunos. Esta aplicação foi projetada para ser intuitiva, segura e eficiente, proporcionando uma experiência de usuário simplificada.

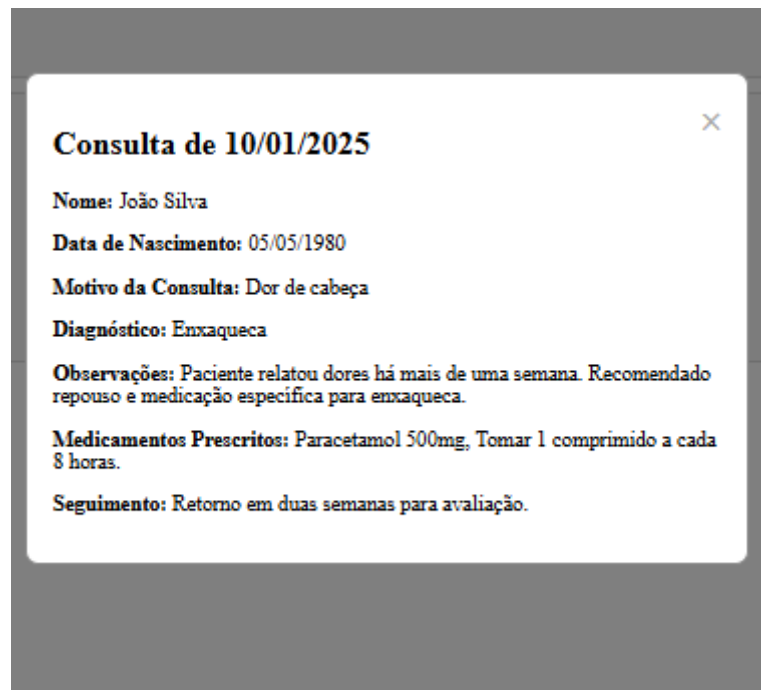


Figura 16 – Tela de visualização do prontuário do paciente

5.1.1. FUNCIONALIDADES PRINCIPAIS

5.1.1.1. ARMAZENAMENTO DIGITAL DE PRONTUÁRIOS

O sistema informatizado permite que os prontuários dos pacientes sejam armazenados de forma totalmente digital, substituindo os registros em papel. Cada prontuário contém informações clínicas detalhadas, como histórico médico, prescrições, resultados de exames, alergias, evolução clínica e dados de consultas anteriores.

5.1.1.2. ACESSO CONTROLADO E SEGURO

Os prontuários digitais são acessíveis apenas a profissionais autorizados mediante autenticação no sistema, como login com senha, autenticação de dois fatores ou biometria. Os acessos são registrados em logs de auditoria, garantindo rastreabilidade e conformidade com normas de proteção de dados, como a LGPD (Lei Geral de Proteção de Dados).

Essa abordagem protege as informações sensíveis dos pacientes e evita acessos não autorizados ou uso indevido dos dados médicos.

5.1.1.3. ATUALIZAÇÃO EM TEMPO REAL

Durante as consultas, os profissionais de saúde podem atualizar os prontuários em tempo real, registrando sintomas, diagnósticos, tratamentos, receitas médicas e outras informações clínicas. Isso melhora a continuidade do cuidado e reduz a chance de erros causados por registros manuais ou informações desatualizadas.

5.1.1.4. HISTÓRICO CLÍNICO COMPLETO

O prontuário digital mantém um histórico clínico completo e cronológico do paciente, permitindo que médicos e outros profissionais de saúde acompanhem a evolução de doenças, tratamentos anteriores e padrões de saúde ao longo do tempo.

5.2. PROCESSO DE IMPLEMENTAÇÃO

5.2.1. ANÁLISE DE REQUISITOS

O desenvolvimento do sistema teve início com uma **análise detalhada dos requisitos**, envolvendo profissionais da saúde, equipe administrativa, corpo técnico de TI e especialistas em segurança da informação do Hospital Militar Central. O principal objetivo foi compreender as necessidades específicas do hospital, os fluxos de trabalho clínico e os desafios relacionados à gestão segura e eficiente de prontuários médicos eletrônicos (PEP).

5.2.2. DESIGN DA APLICAÇÃO

O design da aplicação priorizou uma interface centrada no usuário, levando em conta a rotina e o contexto hospitalar militar. Buscou-se garantir agilidade no uso, clareza visual e robustez na navegação.

Foram utilizados protótipos de interface (UI) com fluxos otimizados para:

- Registro e consulta de histórico clínico.
- Prescrição eletrônica.
- Registro de exames e evolução clínica.

O design foi validado com profissionais de saúde por meio de prototipagem interativa, respeitando princípios de usabilidade em ambientes de alta pressão e com foco na redução de erros operacionais.

5.2.3. DESENVOLVIMENTO E TESTES

A aplicação foi desenvolvida seguindo práticas ágeis (Scrum), com entregas contínuas e feedback iterativo das equipes médicas e administrativas.

Tecnologias utilizadas:

- Backend: PHP com MySQL como banco de dados relacional.
- Frontend: Web responsivo para desktop e tablets, com suporte para navegadores certificados no ambiente militar.
- Integrações: APIs HL7/FHIR para compatibilidade com sistemas de saúde.

Fases de testes:

- Testes unitários e de integração para validação técnica de funcionalidades.
- Testes de aceitação (UAT) com médicos, enfermeiros e administradores do hospital.
- Testes de desempenho para garantir funcionamento estável mesmo em horários de pico.

5.2.4. SEGURANÇA DA INFORMAÇÃO

A segurança foi uma prioridade durante todo o processo de desenvolvimento. Implementamos protocolos de criptografia para proteger os dados dos usuários e garantir transações seguras. Adicionalmente, a aplicação foi desenvolvida para estar em conformidade com as regulamentações de proteção de dados pessoais.

5.2.5. IMPLANTAÇÃO

Após os testes bem-sucedidos, a aplicação foi implantada nas lojas de aplicativos para dispositivos Android e iOS. Também foram realizadas sessões de treinamento para os profissionais de saúde e a equipe administrativa para assegurar uma transição suave e eficiente para o novo sistema.

5.3. BENEFÍCIOS DA APLICAÇÃO

- **Confiabilidade e rapidez no atendimento:** acesso imediato ao histórico clínico, sem necessidade de documentos físicos.
- **Eficiência administrativa:** centralização das informações, redução de papel e automatização de processos.
- **Segurança da informação:** Garante transações seguras através de protocolos de criptografia e conformidade com regulamentações de proteção de dados.
- **Melhoria na qualidade assistencial:** decisões clínicas mais assertivas com base em histórico completo, exames anteriores e interações medicamentosas.

5.4. ANÁLISE E DESENHO DO SISTEMA

Para execução deste projecto foi feito a recolha de dados e análise dos dados, que permitiu o desenho dos principais módulos do sistema e as suas funcionalidades.

5.5. ANÁLISE DE REQUISITOS

Este sistema tem como objetivo primordial fornecer uma experiência melhor aos médicos e pacientes e outros beneficiados pelo mesmo.

Durante esta análise, exploraremos os requisitos funcionais e não funcionais essenciais para o desenvolvimento do sistema, bem como os aspectos de segurança, usabilidade e desempenho que são fundamentais para garantir uma experiência de usuário eficaz e segura.

5.5.1. REQUISITOS FUNCIONAIS

Código	Requisito Funcional
RF 01	Cadastro e consulta de pacientes
RF 02	Registro de consultas, exames, medicamentos e diagnósticos
RF 03	Emissão de relatórios clínicos
RF 04	Acesso simultâneo por profissionais autorizados
RF 05	Cancelar consulta
RF 06	Terminar sessão

Tabela 1 - Requisitos funcionais do aplicativo

5.5.2. REQUISITOS NÃO FUNCIONAIS

Código	Requisito Não Funcional	Categoria
RNF 01	Autenticação de usuário	Segurança
RNF 02	Encriptar senha	Segurança
RNF 03	backup constante de dados para evitar perdas de informações importantes	Segurança
RNF 04	Interface intuitiva e fácil de usar para os recepcionistas e Diretor clínico	Usabilidade

RNF 05	Os tempos de resposta do sistema devem ser rápidos, garantindo uma experiência fluida para o usuário	Desempenho
RNF 06	Escalabilidade horizontal	Desempenho
RNF 07	Facilidade de atualização e manutenção do sistema	Manutenção

Tabela 2 - Requisitos não funcionais identificados

5.6. DIAGRAMA DE CASOS DE USO

Os casos de uso representam um conjunto de ações realizadas para a execução de funcionalidades específicas do sistema de gestão de prontuários eletrônicos. Eles descrevem como os usuários interagem com o sistema para alcançar determinados objetivos, focando-se na relação entre os atores (usuários) e as funções oferecidas.

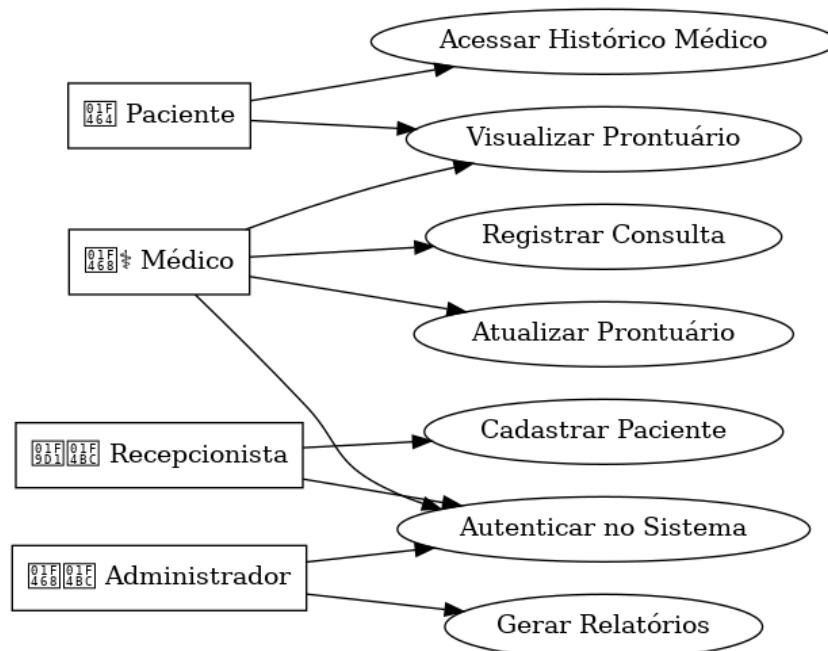


Figura 17 – Diagrama de caso de uso do Sistema

5.7. BASE DE DADOS

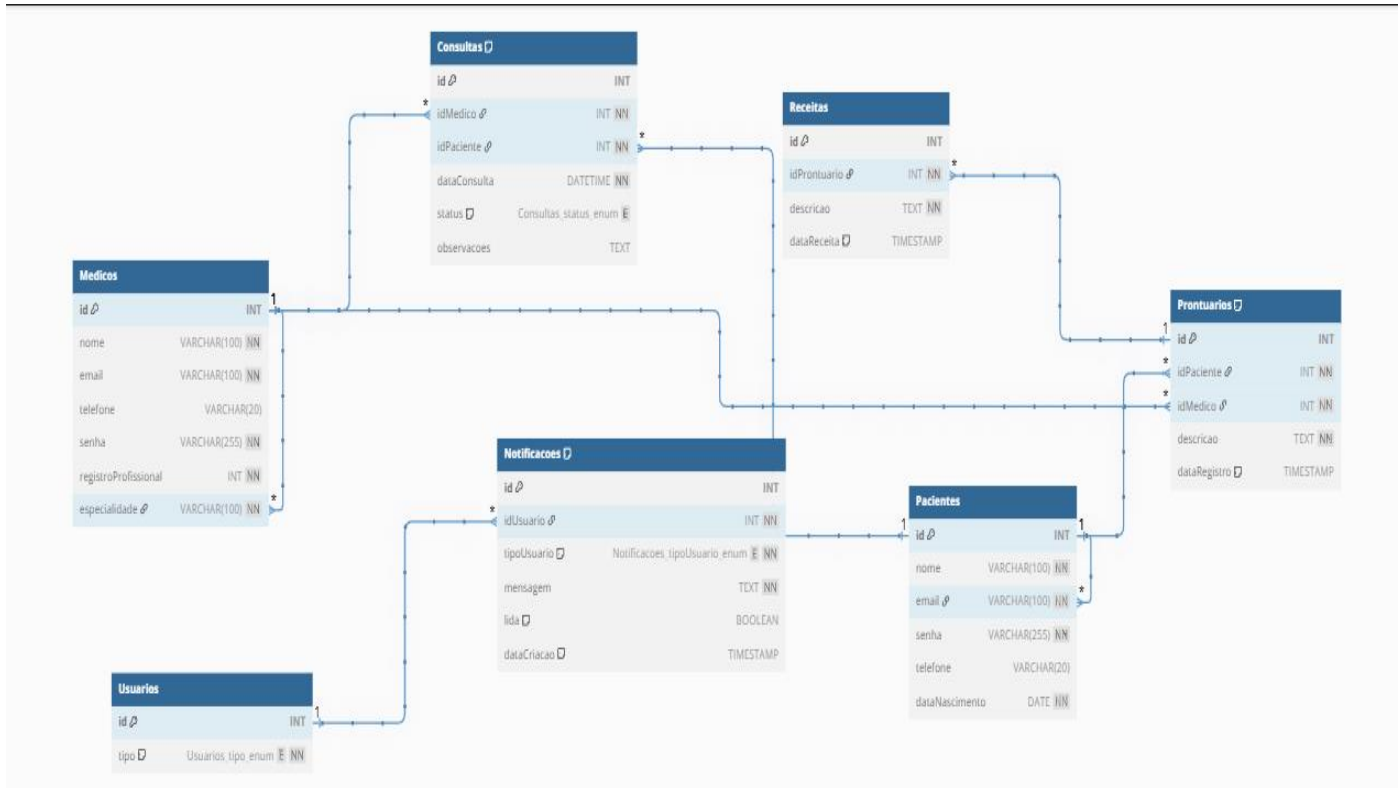


Figura 18 – Diagrama da base de dados

5.7.1. Descrições Principais

generator client {

 provider = "prisma-client-js"

}

datasource db {

 provider = "mysql"

 url = env("DATABASE_URL")

}

5.7.1.1. Médicos

- Guarda as informações dos médicos cadastrados no sistema, como nome, e-mail, telefone, senha, registro profissional (ex: CRM) e especialidade.

- Usada para autenticação e identificação dos médicos que realizam consultas e registram prontuários.

```
CREATE TABLE Medicos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  telefone VARCHAR(20),  
  senha VARCHAR(255) NOT NULL,  
  registroProfissional INT NOT NULL,  
  especialidade VARCHAR(100) NOT NULL  
);
```

5.7.1.2. Pacientes

- Armazena os dados dos pacientes, incluindo nome, e-mail, senha, telefone e data de nascimento.

- Serve para identificar os pacientes que agendam consultas, recebem atendimento e têm seus prontuários registrados.

```
CREATE TABLE Pacientes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  senha VARCHAR(255) NOT NULL,  
  telefone VARCHAR(20),  
  dataNascimento DATE NOT NULL  
);
```

5.4.1.3 Consultas:

- Registra os agendamentos de consultas entre médicos e pacientes.
- Contém a data da consulta, o status (Agendada, Realizada, Cancelada) e observações relacionadas à consulta.
- Serve para controlar e acompanhar os atendimentos realizados no sistema.

```
CREATE TABLE Consultas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  idMedico INT NOT NULL,  
  idPaciente INT NOT NULL,  
  dataConsulta DATETIME NOT NULL,  
  status ENUM('Agendada', 'Realizada', 'Cancelada') DEFAULT 'Agendada',  
  observacoes TEXT,  
  FOREIGN KEY (idMedico) REFERENCES Medicos(id) ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (idPaciente) REFERENCES Pacientes(id) ON DELETE CASCADE ON UPDATE CASCADE  
);
```

5.7.1.4. Prontuarios

- Armazena os registros médicos detalhados feitos pelo médico para cada paciente.
- Contém a descrição do atendimento, exames, diagnósticos e outras informações clínicas relevantes.
- Funciona como o histórico médico do paciente dentro do sistema.

```
CREATE TABLE Prontuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  idPaciente INT NOT NULL,  
  idMedico INT NOT NULL,  
  descricao TEXT NOT NULL,  
  dataRegistro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (idPaciente) REFERENCES Pacientes(id) ON DELETE CASCADE ON UPDATE CASCADE,  
  FOREIGN KEY (idMedico) REFERENCES Medicos(id) ON DELETE CASCADE ON UPDATE CASCADE  
);
```

5.7.1.5. Notification

- Armazena as mensagens e alertas enviados para médicos ou pacientes.
- Cada notificação é vinculada a um usuário (médico ou paciente), contém o texto da mensagem, o status de leitura (lida ou não) e a data de criação.
- Serve para comunicar informações importantes, como lembretes de consulta, alertas ou avisos administrativos.

```
CREATE TABLE Notificacoes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  idUsuario INT NOT NULL,  
  tipoUsuario ENUM('Medico', 'Paciente') NOT NULL,  
  mensagem TEXT NOT NULL,  
  lida BOOLEAN DEFAULT FALSE,  
  dataCriacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (idUsuario) REFERENCES Usuarios(id) ON DELETE CASCADE ON UPDATE CASCADE  
);
```


5.7.1.6 USER(Usuarios)

- Tabela genérica que unifica os médicos e pacientes para facilitar o gerenciamento das notificações.
- Cada registro indica se o usuário é um médico ou paciente.
- Usada para referenciar notificações sem duplicar a lógica para os dois tipos de usuários.

```
- CREATE TABLE Usuarios (  
-     id INT AUTO_INCREMENT PRIMARY KEY,  
-     tipo ENUM('Medico', 'Paciente') NOT NULL  
- );
```

5.7.1.7 Receitas

- Guarda as receitas médicas prescritas pelo médico durante as consultas.
- Cada receita está vinculada a um prontuário específico.
- Contém a descrição da medicação ou tratamento recomendado e a data da receita.

```
- CREATE TABLE Receitas (  
-     id INT AUTO_INCREMENT PRIMARY KEY,  
-     idProntuario INT NOT NULL,  
-     descricao TEXT NOT NULL,  
-     dataReceita TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
-     FOREIGN KEY (idProntuario) REFERENCES Prontuarios(id) ON DELETE CASCADE ON  
-     UPDATE CASCADE  
- );
```

5.7.1.7 EstatisticasConsultas (VIEW)

```
CREATE VIEW EstatisticasConsultas AS  
SELECT  
    MONTH(dataConsulta) AS mes,  
    YEAR(dataConsulta) AS ano,  
    COUNT(*) AS totalConsultas  
FROM Consultas  
WHERE status = 'Realizada'  
GROUP BY ano, mes;
```

```
CREATE INDEX idx_consultas_medico ON Consultas(idMedico);  
CREATE INDEX idx_consultas_paciente ON Consultas(idPaciente);  
CREATE INDEX idx_notificacoes_usuario ON Notificacoes(idUsuario);  
CREATE INDEX idx_prontuarios_paciente ON Prontuarios(idPaciente);
```

5.8. NORMALIZAÇÃO

As formas normais em bancos de dados são regras que definem a estrutura de como os dados devem ser organizados para evitar redundância e anomalias na manipulação dos mesmos. Existem várias formas normais, sendo as mais comuns:

Primeira Forma Normal (1NF): Todos os atributos de uma tabela devem conter valores atômicos, ou seja, valores indivisíveis. Além disso, cada coluna deve conter valores de um único tipo de dado.

Segunda Forma Normal (2NF): A tabela deve estar na 1NF e todos os seus atributos não-chave devem depender completamente da chave primária. Em outras palavras, não pode haver dependência parcial dos atributos não-chave em relação à chave primária.

Terceira Forma Normal (3NF): A tabela deve estar na 2NF e não deve haver dependências transitivas, ou seja, nenhum atributo não-chave deve depender de outro atributo não-chave.

De acordo com estas informações podemos dizer que a nossa base de dados está definida até a 3ª Forma Normal.

5.9. PROTÓTIPO DO SISTEMA



Figura 19 – Protótipo do Sistema

5.10. TECNOLOGIAS USADAS

Para o desenvolvimento do sistema foram usadas as seguintes tecnologias:

5.10.1. SQL

SQL (Structure Query Language) “*é um conjunto de comandos de manipulação de banco de dados utilizado para criar e manter a estrutura desse banco de dados, além de incluir, excluir, modificar e pesquisar informações nas tabelas dele*”. A linguagem SQL não é uma linguagem de programação autônoma; poderia ser chamada de “sublinguagem”. Quando se escrevem aplicações para banco de dados, é necessário utilizar uma linguagem de programação tradicional (C, JAVA, Pascal, COBOL, etc.) e embutir comandos SQL para manipular os dados (PODEROSO DE OLIVEIRA, 2002).



Figura 20 - Logo do SQL

Em um modelo relacional, apenas um tipo de estrutura de dados existe, a “tabela”. Novas tabelas são criadas com a junção ou combinação de outras tabelas. Utilizando apenas um comando SQL é possível pesquisar dados em diversas tabelas ou atualizar e excluir diversas linhas de tabelas (PODEROSO DE OLIVEIRA, 2002).

A linguagem SQL é dividida nos seguintes componentes:

DDL (Data Definition Language) *permite a criação dos componentes do banco de dados, como tabelas, índices, etc.*

Principais comandos DDL:

- ★ CREATE TABLE;
- ★ ALTER TABLE;

- ★ DROP TABLE;
- ★ CREATE INDEX;
- ★ DROP INDEX.

DML (Data Manipulation Language) *permite a manipulação dos dados armazenados no banco de dados.*

Comandos DML:

- ★ INSERT;
- ★ DELETE;
- ★ UPDATE.

DQL (Data Query Language) *permite extrair dados do banco de dados.*

Comando DQL:

- ★ SELECT.

DCL (Data Control Language) *provê a segurança interna do banco de dados.*

Comandos DCL:

- ★ CREATE USER;
- ★ ALTER USER;
- ★ GRANT;
- ★ REVOKE;
- ★ CREATE SCHEMA.

5.10.2. JavaScript

É uma linguagem de programação de alto nível, interpretada e multi paradigma. Ela é conhecida por sua ampla utilização no desenvolvimento web, sendo a principal linguagem de programação dos navegadores modernos. JavaScript é uma linguagem dinâmica, o que significa que os tipos de dados são determinados em tempo de execução, e também é orientada a objetos, permitindo a manipulação de objetos e herança de protótipos.



Figura 21 - Logo do JavaScript

5.10.3. React Native

É um framework para construir aplicativos móveis usando JavaScript e React. Ele permite que os desenvolvedores criem aplicativos nativos para iOS e Android usando o mesmo código base. Em vez de usar elementos de interface do usuário tradicionais, como UIViews no iOS e Views no Android, o React Native utiliza componentes React que são mapeados para componentes nativos. Isso permite que os aplicativos criados com React Native tenham desempenho semelhante aos aplicativos nativos, enquanto compartilham uma grande parte do código entre as plataformas.

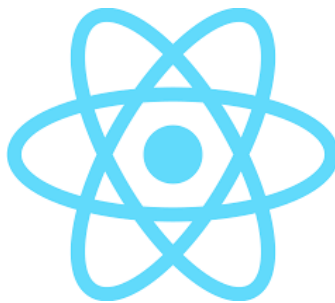


Figura 22 - Logo React Native

5.10.4. CSS (Cascading Style Sheets)

É a linguagem de estilo utilizada para definir a aparência e o layout dos elementos HTML no navegador. Juntamente com o HTML e o JavaScript, o CSS é uma parte fundamental do desenvolvimento web moderno. No contexto do seu sistema, o CSS provavelmente foi utilizado para estilizar os componentes e proporcionar uma experiência visualmente atraente aos usuários.

5.10.7. Node Js

É um ambiente de execução JavaScript assíncrono e baseado em eventos, projetado para criar aplicativos de rede escaláveis. Ele permite que os desenvolvedores usem JavaScript no lado do servidor para construir aplicativos web e APIs. Node.js é conhecido por sua eficiência e escalabilidade, sendo amplamente utilizado em empresas de todos os tamanhos, desde startups até grandes corporações.



Figura 23 - Logo do Node Js

5.10.8. Express.js

É uma linguagem interpretada livre, usada originalmente apenas para o desenvolvimento de aplicações presentes e atuantes no lado do servidor, capazes de gerar conteúdo dinâmico na World Wide Web. Para desenvolver em PHP, um servidor web completo em execução em sua máquina se faz necessário. Ou seja, é preciso um servidor PHP, HTTP, banco de dados e um editor para codificar. Criado por Rasmus Lerdorf em 1995, o PHP tem a produção de sua implementação principal, referência formal da linguagem, mantida por uma organização chamada The PHP Group. O PHP é software livre, licenciado sob a PHP License, uma licença incompatível com a GNU General Public License (GPL) devido a restrições no uso do termo PHP



Figura 24 - Logo do PHP

5.10.9. MySQL

É um sistema de gerenciamento de banco de dados relacional (RDBMS) de código aberto. Ele utiliza a linguagem SQL (Structured Query Language) para manipular e gerenciar dados em bancos de dados relacionais. MySQL é conhecido por sua confiabilidade, escalabilidade e desempenho, sendo amplamente utilizado em aplicativos web para armazenar e recuperar dados de forma eficiente.



Figura 25 - Logo do MySql

5.10.10. Outras tecnologias

Além das Tecnologias em destaque tiveram outras que também puderam auxiliar naquilo que é a execução do projeto. Dentre elas:

5.10.10.1. VScode (Visual Studio Code)

Um editor de código rápido, leve e altamente personalizável da Microsoft.



Figura 26 - Logo do Node Js

5.10.10.2. JSX (JavaScript XML)

Uma extensão de sintaxe do JavaScript usada com o React para criar interfaces de

usuário declarativas.



Figura 27 - Logo do JSX

5.10.10.3. Axios

Uma biblioteca JavaScript para fazer solicitações HTTP a partir do navegador ou do Node.js, oferecendo uma interface fácil de usar e suporte a interceptadores de solicitação e resposta.

5.10.10.4. Npm (Node Package Manager)

O gerenciador de pacotes padrão para o Node.js, usado para instalar e gerenciar dependências de projetos JavaScript.

5.10.10.5. WAMP (Windows, Apache, MySQL, PHP/Python/Perl)

Uma pilha de software para ambiente de desenvolvimento web em sistemas Windows, incluindo Apache, MySQL e PHP/Python/Perl.



Figura 28 - Logo do WAMP

5.11. PADRÕES DE ARQUITETURAS DE SOFTWARES

5.11.1. MVC

Model View Controller (MVC), “*é um padrão de arquitetura de software (não confundir com um Design Pattern) que separa a representação da informação da interação do usuário com ela*”. É normalmente usado para o desenvolvimento de interfaces de usuário que divide uma aplicação em três partes interconectadas. Isto é feito para separar representações de informação internas dos modos como a informação é apresentada para e aceita pelo usuário. (REENSKAUG e COPLIEN, 2009).

O padrão de projeto MVC separa estes componentes maiores possibilitando a reutilização de código e desenvolvimento paralelo de maneira eficiente:

- 1) *Model (Modelo)*: consiste nos dados da aplicação, regras de negócios, lógica e funções.
- 2) *View (Visão)*: pode ser qualquer saída de representação dos dados, como uma tabela ou um diagrama. É possível ter várias visões do mesmo dado, como um gráfico de barras para gerenciamento e uma visão tabular para contadores.
- 3) *Controller (Controlador)*: faz a mediação da entrada, convertendo-a em comandos para o modelo ou visão. As ideias centrais por trás do MVC são a reusabilidade de código e separação de conceitos (REENSKAUG e COPLIEN, 2009).



CAPITULO VI

RESULTADOS E RECOMENDAÇÕES



6. RESULTADOS E RECOMENDAÇÕES

6.1 RESULTADOS

A seguir, são apresentadas imagens de diversas partes da interface do sistema, proporcionando uma visão dos resultados alcançados com o desenvolvimento. Aqui estão as imagens:

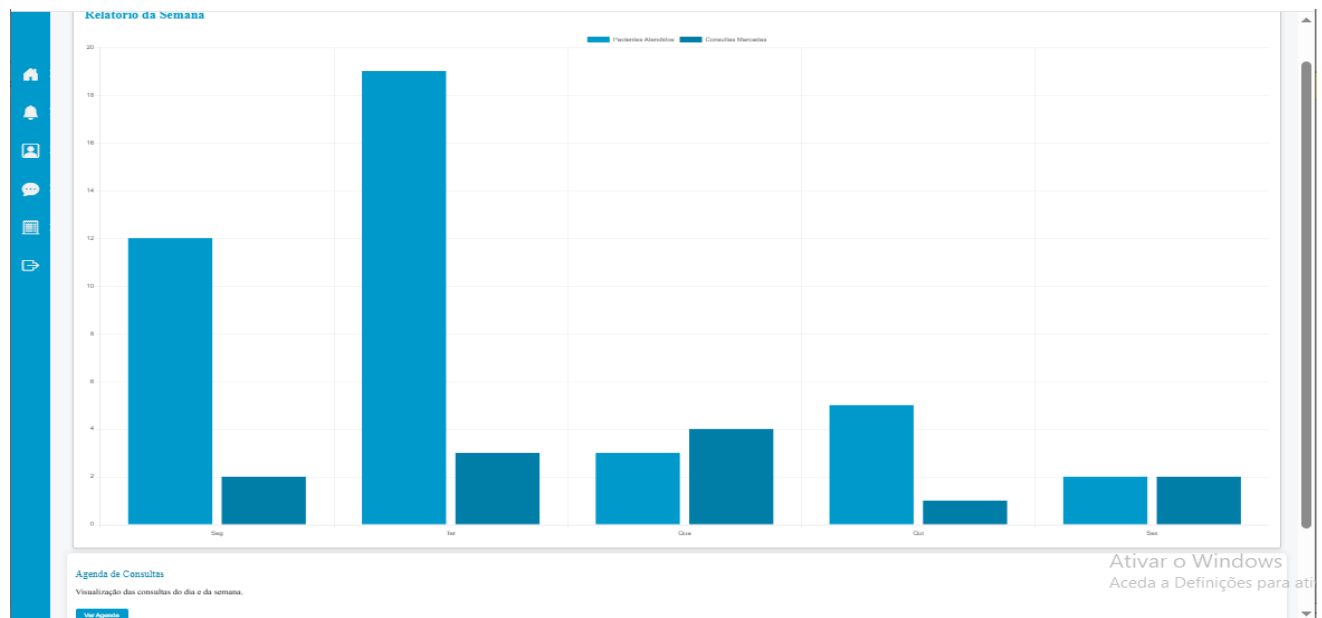


Figura 29 – Página inicial para médicos

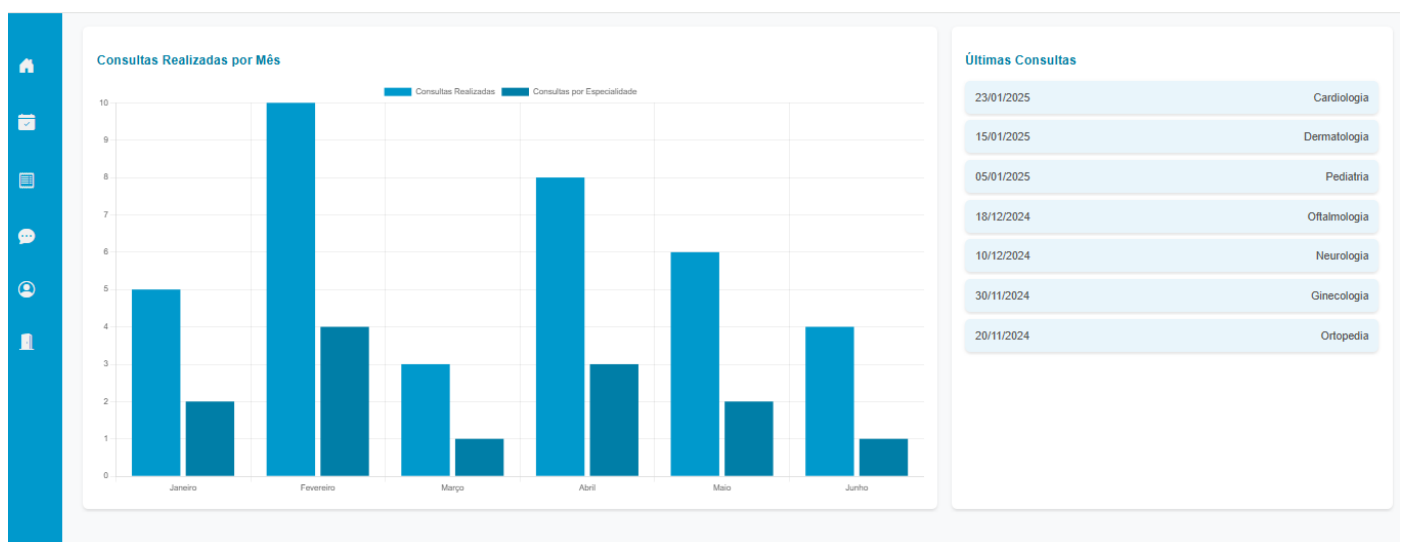


Figura 30 – Página inicial para pacientes

6.2. RECOMENDAÇÕES

Recomenda-se os seguintes requisitos mínimos para o bom funcionamento da aplicação:

- ★ Conexão com a internet;
- ★ Qualquer Sistema operacional (de preferência Windows 10 ou 11)
- ★ Verificar se o computador tem RAM suficiente para a execução da aplicação de forma agradável e consistente.
- ★ Navegador atualizado como: Google Chrome, Mozilla Firefox, Safari, Microsoft Edge;
- ★ Utilizar conexão boa com a internet e com velocidade agradável para evitar latência e carregamentos longos.

As recomendações listadas acima devem ser seguidas para o bom uso da aplicação de forma consistente e agradável.

DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto, enfrentei várias dificuldades que afetaram o ritmo de progresso. Abaixo, detalhamos algumas dessas dificuldades:

Computadores: A falta de computadores adequados foi um obstáculo significativo, pois apenas um dos dispositivos que eu utilizava tinha recursos suficientes para suportar os programas necessários ao desenvolvimento do software. Com o tempo, consegui adquirir mais computadores para dar suporte às atividades e ampliar a estrutura de trabalho.

Internet: A internet foi crucial para o desenvolvimento do sistema, sendo indispensável tanto para a configuração inicial quanto para a construção, pesquisa, instalação de novas dependências e consumo de serviços externos. A necessidade de conexão constante representou um custo elevado, pois precisava de internet diariamente.

Alimentação: Desenvolver a aplicação na escola muitas vezes deixava-me sem condições adequadas de alimentação. Muitas vezes, não alimentava-me adequadamente, o que afetava o meu desempenho lógico e o progresso do projeto.

Hospedagem: Usei diferentes plataformas para hospedar a minha aplicação, optando inicialmente por pacotes gratuitos. Isso resultou em várias limitações, como lentidão nas requisições e, ocasionalmente, indisponibilidade da própria plataforma de hospedagem.

Recursos limitados: Os testes contínuos do sistema exigiram consideráveis recursos financeiros.

Esses desafios foram superados com planejamento, esforço e adaptações ao longo do projeto, permitindo-me continuar e concluir o desenvolvimento do sistema.

CONCLUSÃO

Ao concluir este trabalho, destaco o esforço que dediquei à implementação de um sistema para o Hospital Militar Central, com o objetivo de modernizar e otimizar os processos de gestão dos prontuários eletrônicos dos pacientes. Durante o desenvolvimento, percebi a importância de melhorar a eficiência operacional, reduzir a carga administrativa do pessoal de saúde e proporcionar uma experiência mais segura e ágil para os profissionais médicos e pacientes.

Enfatizei a segurança da informação como uma preocupação central, reconhecendo a necessidade de proteger os dados médicos dos pacientes contra ameaças cibernéticas e garantir a conformidade com as normas de privacidade e sigilo médico. Além disso, valorizei a automação dos processos para aumentar a eficiência, eliminando tarefas manuais e agilizando o acesso, a atualização e o arquivamento dos prontuários.

Ao longo deste trabalho, constatei que a implementação bem-sucedida deste sistema promoverá uma gestão hospitalar mais eficiente no Hospital Militar Central e trará um impacto positivo direto na qualidade do atendimento, oferecendo um ambiente mais seguro, conveniente e rápido para o acesso às informações clínicas.

Portanto, concluo que a implementação deste sistema representa um passo significativo na modernização das práticas administrativas e clínicas do Hospital Militar Central, refletindo o meu compromisso com a excelência no cuidado à saúde e a proteção dos direitos dos pacientes.

GLOSSÁRIO DE TERMOS

Software: Programa de computador ou conjunto de programas que instruem um computador a realizar tarefas específicas. O software pode ser categorizado em software de sistema, que fornece funcionalidades de baixo nível para suportar o funcionamento do computador, e software de aplicativo, que é desenvolvido para realizar tarefas específicas para os usuários.

Figma: Ferramenta de design de interface do usuário (UI) baseada em nuvem, que permite que equipes de design e desenvolvimento criem, compartilhem e colaborem em projetos de design de forma eficiente. Figma oferece recursos para prototipagem, design de interfaces, colaboração em tempo real e geração de código.

API (Interface de Programação de Aplicativos): Conjunto de definições e protocolos que permite que diferentes softwares se comuniquem entre si. As APIs fornecem uma interface para que os desenvolvedores acessem e interajam com os recursos ou funcionalidades de um sistema ou serviço, geralmente por meio de solicitações HTTP.

Online: Refere-se a algo que está conectado à internet ou disponível através dela. Quando algo está "online", está acessível e disponível para acesso e interação remotos, permitindo que os usuários o acessem de qualquer lugar com uma conexão à internet.

UI (Interface do Usuário): A parte de um software ou sistema que permite que os usuários interajam com ele. A UI inclui elementos visuais, como botões, menus, campos de entrada e layouts de tela, que permitem que os usuários naveguem e operem o software de forma eficaz.

APÊNDICE

APÊNDICE A – TABELA DO CRONOGRAMA DAS ACTIVIDADES

Tarefas	Duração	Início	Término	Prodecessores
Escolha do Tema	2 sem.	Seg 18/09/24	Sext 29/09/24	-
Recolha das informações	2 sem.	Seg 02/10/24	Sext 13/10/24	1
Entrega do Relatório	1 mês	Qui 11/04/25	Seg 13/05/25	6,7
Desenho da UI da aplicação	6 meses	Seg 05/11/25	Sex 12/04/25	2
Construção da BD	2 dias	Qua 08/11/24	Qui 09/11/25	8
Construção do Back-End da aplicação	5 meses	Seg 11/12/24	Sex 05/04/25	5
Construção do Front-End da aplicação	4 meses	Seg 08/01/25	Sex 05/04/25	4
Modelagem da BD	1 sem.	Ter 05/12/24	Ter 12/12/25	2
Teste	1 sem	Qui 02/05/25	Qui 09/05/25	6,7

Tabela 3 - Tabela do cronograma de atividades

APÊNDICE B – ORÇAMENTO

Nº	DESCRIÇÃO	CATEGORIA	PREÇO	TOTAL
01	Alimentação	Necessidade	55.000,00	55.000,00
02	Analista de sistema	RH	0,00	0,00
03	Programadores	RH	0,00	0,00
04	Computadores	Recurso material	150.000,00	300.000,00
05	Internet	Recurso de Pesquisa	30.000,00	30.000,00
06	Transporte	Necessidade	49.000,00	49.000,00
07	Impressão do relatório	Recurso didático	6.000,00	6.000,00
08	Capa dura	Recurso didático	10.000,00	10.000,00
09	Ternos sociais	Recurso material	100.000,00	100.000,00
TOTAL				550.000,00 KZ

Tabela 4 – Orçamento do projecto

APÊNDICE C – FORMULÁRIOS



Cadastro de Médicos

Nome Completo

Email

Telemóvel

Número de Registro Profissional

Senha

Especialidade

Cadastrar

[Já tem uma conta? Faça login](#)

Figura 31 – Tela de Cadastro



Iniciar secção de Médicos

Nome Completo

Email

Número de Registro Profissional

Senha

Iniciar secção

[Não tem uma conta? Crie uma](#)

Figura 32 – Tela de Login

Consultas Agendadas

Paciente	Data	Hora	Status	Ações
João Silva	10/01/2025	14:00	Agendada	Atualizar Status Adicionar Nota
Maria Oliveira	12/01/2025	10:00	Concluída	Atualizar Status Adicionar Nota

Agendar Nova Consulta

[Agendar Consulta](#)

Figura 33 – Tela de Agendamento de consulta

Meu Perfil

Nome:

Senha:

Especialidade:

Email:

Telefone:

Prestador:

[Editar Perfil](#)
[Salvar](#)

Informações do medico

Nome do medico Stélvio Ribeiro

Especialidade: Cardiologista

E-mail: jose.santos@hospital.com

Prestador:É Prestador

Figura 34 – Tela de perfil

REFERÊNCIAS BIBLIOGRÁFICAS

1. BRASIL. *Ministério da Saúde*. Prontuário Eletrônico do Paciente: aspectos legais, técnicos e éticos. Brasília: Ministério da Saúde, 2020.
2. SOBRINHO, D. F. S. et al. *Sistemas de informação em saúde: fundamentos e aplicações*. São Paulo: Editora Manole, 2019.
3. CUNHA, M. A.; CUNHA, R. R. *Gestão da informação em saúde: prontuário eletrônico e seus desafios*. Rio de Janeiro: Elsevier, 2018.
4. SILVA, M. R. da; PEREIRA, A. C. *Informática em Saúde: fundamentos para a formação de profissionais da saúde*. São Paulo: Editora Atheneu, 2021.
5. DIAS, C. M. T.; GOMES, D. C. *Tecnologia da informação na saúde: prontuário eletrônico e segurança de dados*. Curitiba: Editora CRV, 2020.
6. LAUDON, K. C.; LAUDON, J. P. *Sistemas de informação gerenciais: gerenciando a empresa digital*. 15. ed. São Paulo: Pearson, 2021.
7. MACHADO, J. P. et al. *Segurança da informação em ambientes hospitalares*. Revista Brasileira de Informática em Saúde, v. 16, n. 2, 2020.
8. OLIVEIRA, T. L. de; NASCIMENTO, M. F. *Privacidade e proteção de dados no prontuário eletrônico do paciente*. Revista Saúde e Desenvolvimento, v. 12, n. 1, 2020.
9. ISO/IEC 27799:2016. *Health informatics — Information security management in health using ISO/IEC 27002*. International Organization for Standardization, 2016.
10. PEREIRA, M. R.; SANTOS, R. F. *Desafios da implantação do prontuário eletrônico em instituições públicas de saúde*. Revista de Gestão em Sistemas de Saúde, v. 9, n. 1, 2020.
11. GOMES, A. C.; LIMA, L. C. *Automatização de processos hospitalares com foco na eficiência e qualidade do atendimento*. São Paulo: Pioneira Thomson Learning, 2019.
12. COLOMBO, A. A. *Gestão hospitalar: teoria e prática*. São Paulo: Atlas, 2022.
13. SANTOS, M. F.; MORAES, H. M. *Tecnologia da informação em hospitais militares: desafios e estratégias de implantação*. Revista Brasileira de Gestão Militar, v. 5, n. 2, 2021.
14. PINHEIRO, R. et al. *Atenção primária e redes de atenção à saúde*. Rio de Janeiro: CEPESC, 2020.
15. FIGUEIREDO, M. G. et al. *Informação em saúde: uma abordagem multidisciplinar*. Belo Horizonte: Nescon/UFMG, 2019.